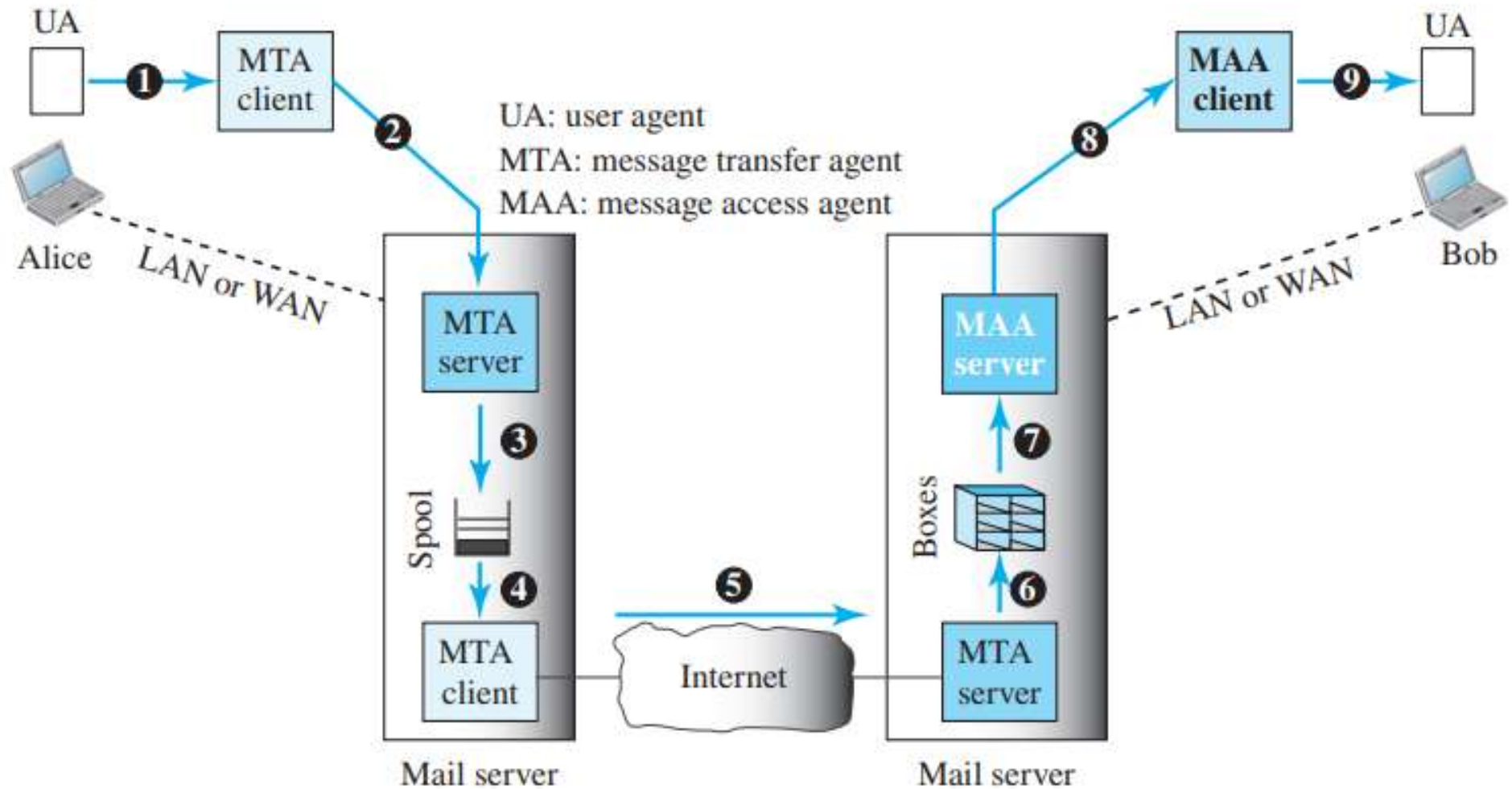


Module No. 6	Application Layer Protocols
Application Layer Protocols: World wide web and HTTP, HTTPS, Domain names: DNS, File Transfer: FTP, Electronic mail: SMTP, Peer to peer networking, Torrent, VPN. Session management, Data compression techniques.	

Electronic Mail (SMTP)

- Electronic mail (e-mail) allows users to exchange messages.
- First, e-mail is considered a one-way transaction.
- When Alice sends an e-mail to Bob, she may expect a response, but this is not a mandate.
- Bob may or may not respond.
- If he does respond, it is another one-way transaction.
- Second, it is neither feasible nor logical for Bob to run a server program and wait until someone sends an e-mail to him.
- Bob may turn off his computer when he is not using it.
- This means that the idea of client/server programming should be implemented in another way: using some intermediate computers (servers).
- The users run only client programs when they want and the intermediate servers apply the client/server paradigm.

Common scenario



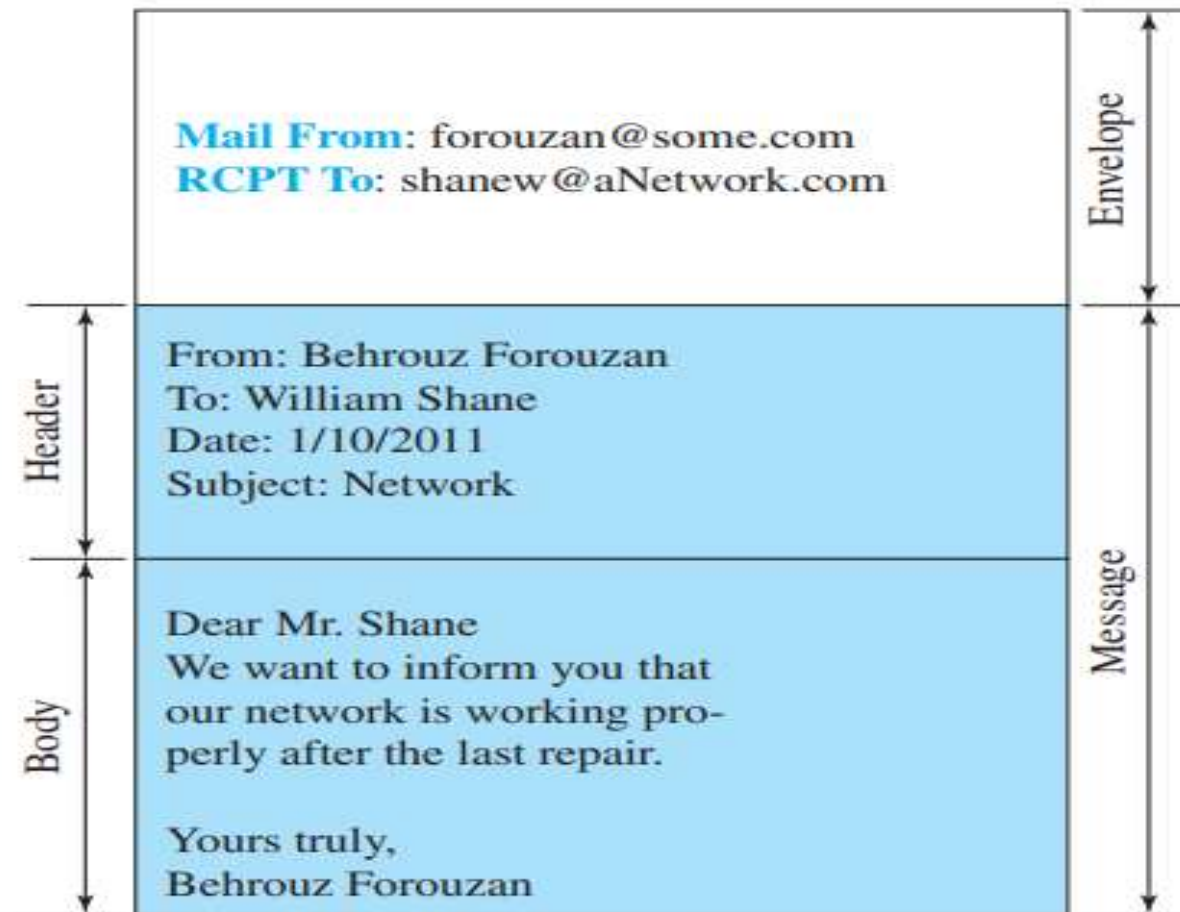
User Agent

- The first component of an electronic mail system is the user agent (UA).
- It provides service to the user to make the process of sending and receiving a message easier.
- A user agent is a software package (program) that composes, reads, replies to, and forwards messages.
- It also handles local mailboxes on the user computers.
- There are two types of user agents: **command-driven** and **GUI-based**.

Sending Mail

- It has an envelope and a message.
- The envelope usually contains the sender address, the receiver address, and other information.
- The message contains the header and the body.
- The header of the message defines the sender, the receiver, the subject of the message, and some other information.
- The body of the message contains the actual information to be read by the recipient.

Format of an e-mail



Electronic mail

Receiving Mail

- The user agent is triggered by the user (or a timer).
- If a user has mail, the **UA informs** the user with a notice.
- If the user is ready to read the mail, a list is displayed in which each line contains a summary of the information about a particular message in the mailbox.
- The summary usually includes the sender mail address, the subject, and the time the mail was sent or received. The user can select any of the messages and display its contents on the screen.

Addresses

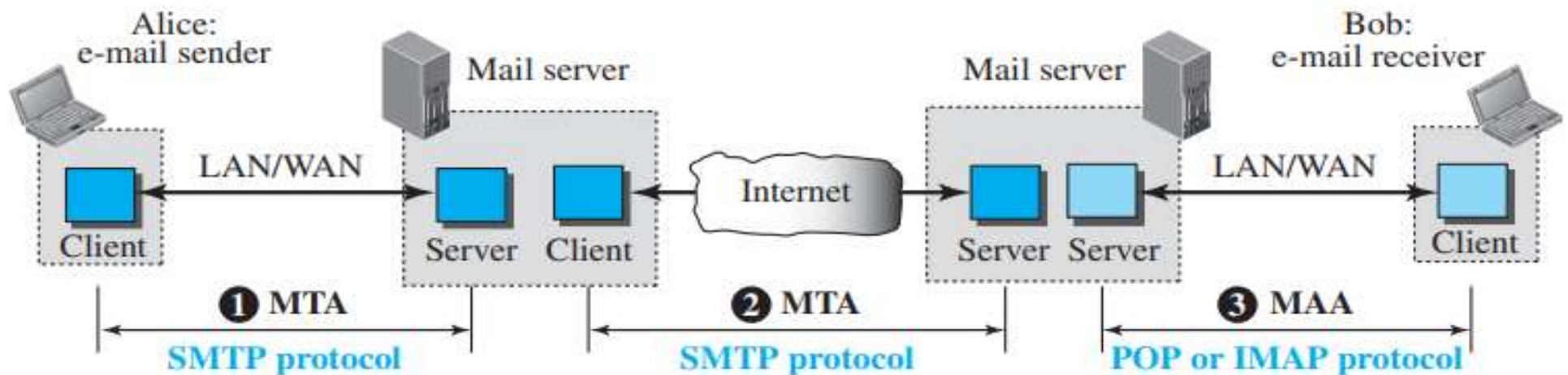
- To deliver mail, a mail handling system must use an addressing system with unique addresses.
- In the Internet, the address consists of two parts: a local part and a domain name, separated by an @ sign.



Mailing List or Group List

- Electronic mail allows one name, an alias, to represent several different e-mail addresses; this is called a mailing list.
- Every time a message is to be sent, the system checks the recipient's name against the alias database; if there is a mailing list for the defined alias, separate messages, one for each entry in the list, must be prepared and handed to the MTA.

Message Transfer Agent: SMTP



Protocols used in electronic mail

- The formal protocol that defines the MTA client and server in the Internet is called **Simple Mail Transfer Protocol (SMTP)**.
- SMTP is used two times, between the sender and the sender's mail server and between the two mail servers.
- SMTP simply defines how commands and responses must be sent back and forth.

Commands and Responses

- SMTP uses commands and responses to transfer messages between an MTA client and an MTA server.
- The command is from an MTA client to an MTA server; the response is from an MTA server to the MTA client.
- Each command or reply is terminated by a two-character (carriage return and line feed) end-of-line token.

Commands

- Commands are sent from the client to the server.

<i>Keyword</i>	<i>Argument(s)</i>	<i>Description</i>
HELO	Sender's host name	Identifies itself
MAIL FROM	Sender of the message	Identifies the sender of the message
RCPT TO	Intended recipient	Identifies the recipient of the message
DATA	Body of the mail	Sends the actual message
QUIT		Terminates the message
RSET		Aborts the current mail transaction
VERFY	Name of recipient	Verifies the address of the recipient
NOOP		Checks the status of the recipient
TURN		Switches the sender and the recipient
EXPN	Mailing list	Asks the recipient to expand the mailing list
HELP	Command name	Asks the recipient to send information about the command sent as the argument
SEND FROM	Intended recipient	Specifies that the mail be delivered only to the terminal of the recipient, and not to the mailbox
SMOL FROM	Intended recipient	Specifies that the mail be delivered to the terminal <i>or</i> the mailbox of the recipient
SMAL FROM	Intended recipient	Specifies that the mail be delivered to the terminal <i>and</i> the mailbox of the recipient

Responses

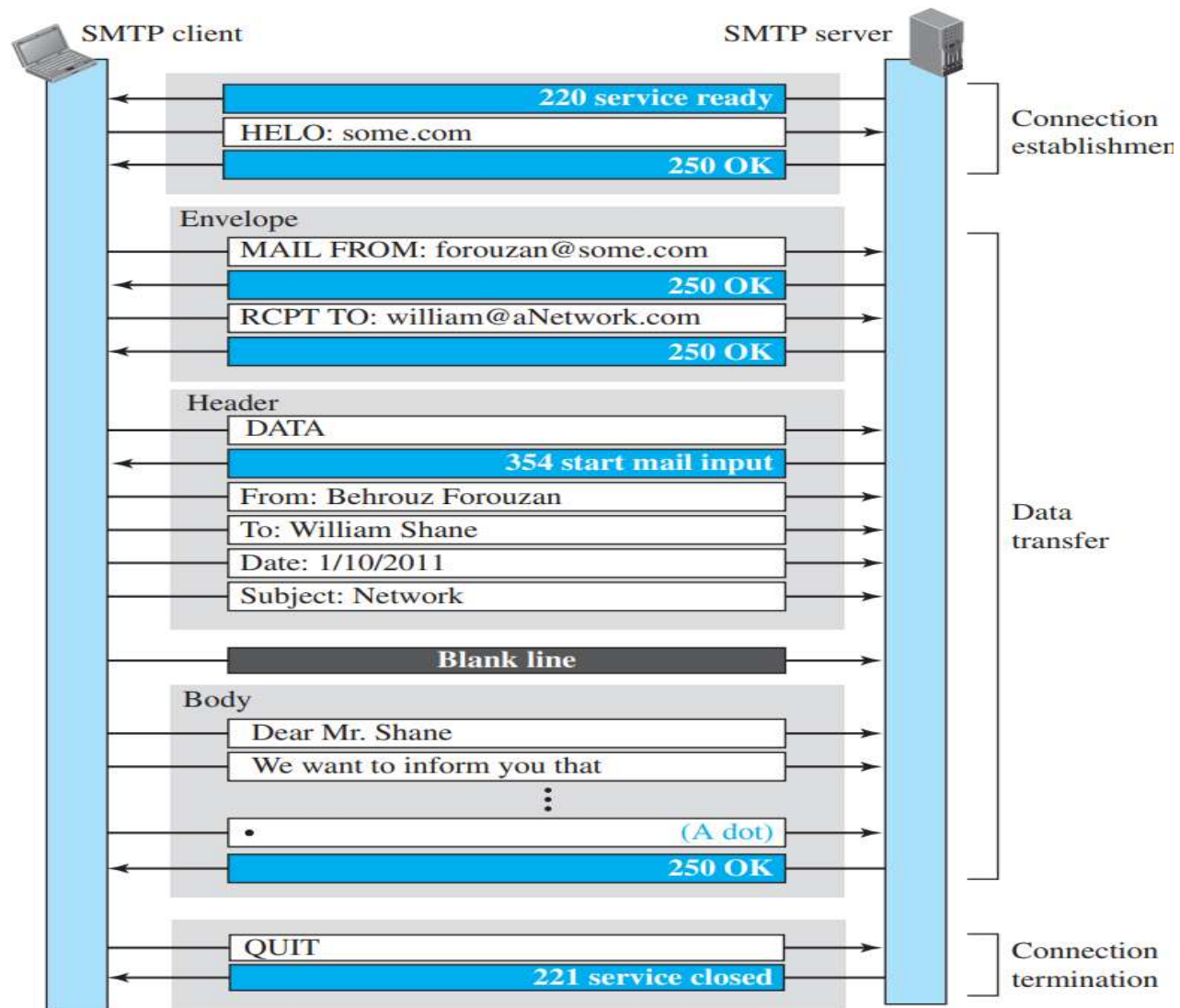
- Responses are sent from the server to the client.
- A response is a three-digit code that may be followed by additional textual information.

<i>Code</i>	<i>Description</i>
Positive Completion Reply	
211	System status or help reply
214	Help message
220	Service ready
221	Service closing transmission channel
250	Request command completed
251	User not local; the message will be forwarded
Positive Intermediate Reply	
354	Start mail input
Transient Negative Completion Reply	
421	Service not available
450	Mailbox not available
451	Command aborted: local error
452	Command aborted; insufficient storage
Permanent Negative Completion Reply	
500	Syntax error; unrecognized command

<i>Code</i>	<i>Description</i>
501	Syntax error in parameters or arguments
502	Command not implemented
503	Bad sequence of commands
504	Command temporarily not implemented
550	Command is not executed; mailbox unavailable
551	User not local
552	Requested action aborted; exceeded storage location
553	Requested action not taken; mailbox name not allowed
554	Transaction failed

Mail Transfer Phases

The process of transferring a mail message occurs in three phases: connection establishment, mail transfer, and connection termination.



Connection Establishment After a client has made a TCP connection to the well-known port 25, the SMTP server starts the connection phase. This phase involves the following three steps:

1. The server sends code 220 (service ready) to tell the client that it is ready to receive mail. If the server is not ready, it sends code 421 (service not available).
2. The client sends the HELO message to identify itself, using its domain name address. This step is necessary to inform the server of the domain name of the client.
3. The server responds with code 250 (request command completed) or some other code depending on the situation.

Message Transfer After connection has been established between the SMTP client and server, a single message between a sender and one or more recipients can be exchanged. This phase involves eight steps. Steps 3 and 4 are repeated if there is more than one recipient.

1. The client sends the MAIL FROM message to introduce the sender of the message. It includes the mail address of the sender (mailbox and the domain name). This step is needed to give the server the return mail address for returning errors and reporting messages.
2. The server responds with code 250 or some other appropriate code.
3. The client sends the RCPT TO (recipient) message, which includes the mail address of the recipient.
4. The server responds with code 250 or some other appropriate code.
5. The client sends the DATA message to initialize the message transfer.
6. The server responds with code 354 (start mail input) or some other appropriate message.
7. The client sends the contents of the message in consecutive lines. Each line is terminated by a two-character end-of-line token (carriage return and line feed). The message is terminated by a line containing just one period.
8. The server responds with code 250 (OK) or some other appropriate code.

Connection Termination After the message is transferred successfully, the client terminates the connection. This phase involves two steps.

1. The client sends the QUIT command.
2. The server responds with code 221 or some other appropriate code.

WORLD WIDE WEB AND HTTP

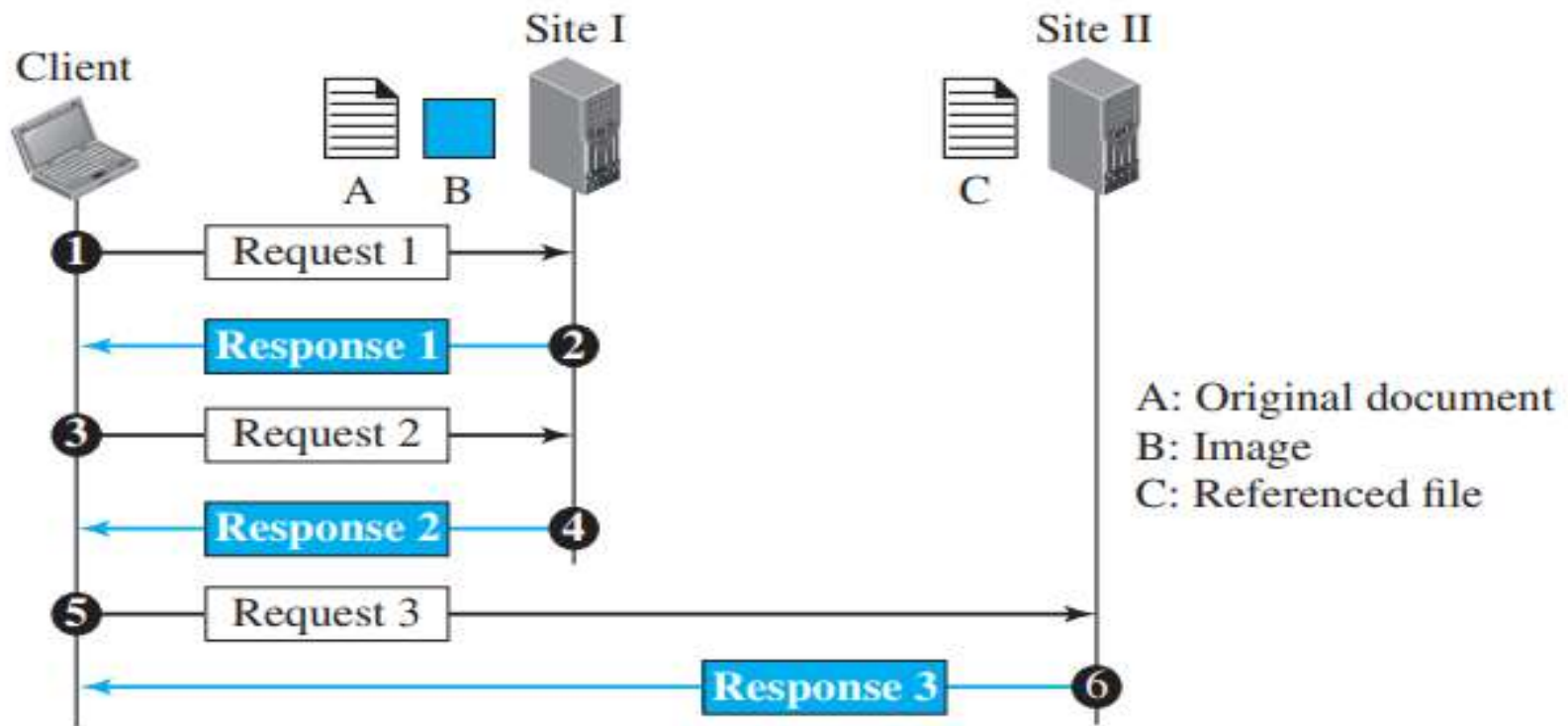
- The Web today is a repository of information in which the documents, called web pages, are distributed all over the world and related documents are linked together.
- Linking allows one web page to refer to another web page stored in another server somewhere else in the world. The linking of web pages was achieved using a concept called **hypertext**.
- Today, the term hypertext, coined to mean linked text documents, has been changed to **hypermedia**, to show that a web page can be a text document, an image, an audio file, or a video file.

Architecture

- The WWW today is a distributed client-server service, in which a client using a browser can access a service using a server. However, the service provided is distributed over many locations called sites.
- Each site holds one or more web pages. Each web page, however, can contain some links to other web pages in the same or other sites.
- In other words, a web page can be simple or composite. A simple web page has no links to other web pages; a composite web page has one or more links to other web pages. Each web page is a file with a name and address.

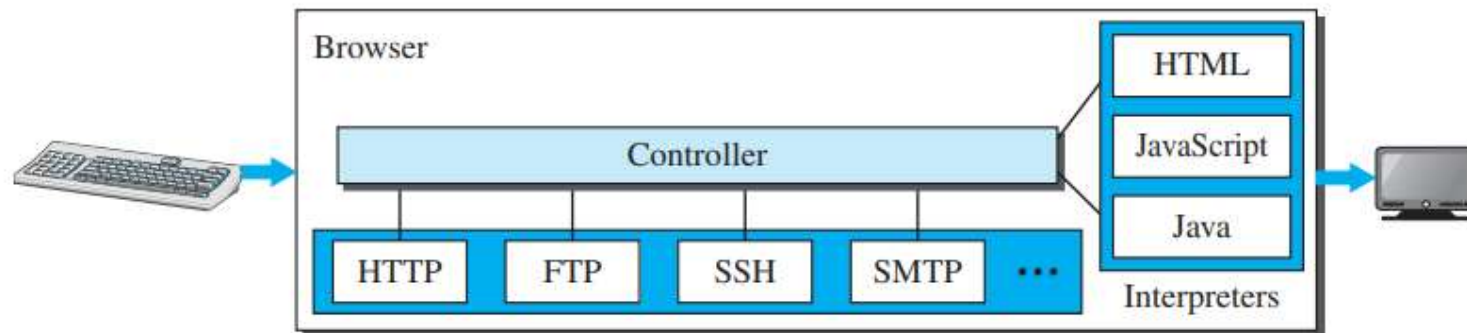
- The main document and the image are stored in two separate files (file A and file B) in the same site; the referenced text file (file C) is stored in another site.
- Since we are dealing with three different files, we need three transactions if we want to see the whole document. The first transaction (request/response) retrieves a copy of the main document (file A), which has references (pointers) to the second and third files.
- When a copy of the main document is retrieved and browsed, the user can click on the reference to the image to invoke the second transaction and retrieve a copy of the image (file B).
- If the user needs to see the contents of the referenced text file, she can click on its reference (pointer) invoking the third transaction and retrieving a copy of file C. Note that although files A and B both are stored in site I, they are independent files with different names and addresses.
- Two transactions are needed to retrieve them. A very important point we need to remember is that file A, file B, and file C in below example are independent web pages, each with independent names and addresses.

- Although references to file B or C are included in file A, it does not mean that each of these files cannot be retrieved independently. A second user can retrieve file B with one transaction.
- A third user can retrieve file C with one transaction.



Web Client (Browser)

- A variety of vendors offer commercial browsers that interpret and display a web page, and all of them use nearly the same architecture.
- Each browser usually consists of three parts: a controller, client protocols, and interpreters.



- The controller receives input from the keyboard or the mouse and uses the client programs to access the document.
- After the document has been accessed, the controller uses one of the interpreters to display the document on the screen.
- The client protocol can be one of the protocols described later, such as HTTP or FTP.
- The interpreter can be HTML, Java, or JavaScript, depending on the type of document. Some commercial browsers include Internet Explorer, Netscape Navigator, and Firefox.

Web Server

- The web page is stored at the server.
- Each time a request arrives, the corresponding document is sent to the client.
- To improve efficiency, servers normally store requested files in a cache in memory; memory is faster to access than a disk.
- A server can also become more efficient through multithreading or multiprocessing.
- In this case, a server can answer more than one request at a time.
- Some popular web servers include Apache and Microsoft Internet Information Server.

Uniform Resource Locator (URL)

A web page, as a file, needs to have a unique identifier to distinguish it from other web pages. To define a web page, we need three identifiers: *host*, *port*, and *path*. However, before defining the web page, we need to tell the browser what client-server application we want to use, which is called the *protocol*. This means we need four identifiers to define the web page. The first is the type of vehicle to be used to fetch the web page; the last three make up the combination that defines the destination object (web page).

- ***Protocol.*** The first identifier is the abbreviation for the client-server program that we need in order to access the web page. Although most of the time the protocol is HTTP (HyperText Transfer Protocol), which we will discuss shortly, we can also use other protocols such as FTP (File Transfer Protocol).

- ❑ **Host.** The host identifier can be the IP address of the server or the unique name given to the server. IP addresses can be defined in dotted decimal notation, as described in Chapter 18 (such as 64.23.56.17); the name is normally the domain name that uniquely defines the host, such as *forouzan.com*, which we discuss in Domain Name System (DNS) later in this chapter.
- ❑ **Port.** The port, a 16-bit integer, is normally predefined for the client-server application. For example, if the HTTP protocol is used for accessing the web page, the well-known port number is 80. However, if a different port is used, the number can be explicitly given.

- ❑ **Path.** The path identifies the location and the name of the file in the underlying operating system. The format of this identifier normally depends on the operating system. In UNIX, a path is a set of directory names followed by the file name, all separated by a slash. For example, */top/next/last/myfile* is a path that uniquely defines a file named *myfile*, stored in the directory *last*, which itself is part of the directory *next*, which itself is under the directory *top*. In other words, the path lists the directories from the top to the bottom, followed by the file name.

To combine these four pieces together, the **uniform resource locator (URL)** has been designed; it uses three different separators between the four pieces as shown below:

protocol://host/path

Used most of the time

protocol://host:port/path

Used when port number is needed

Web Documents

The documents in the WWW can be grouped into three broad categories: static, dynamic, and active.

Static Documents

Static documents are fixed-content documents that are created and stored in a server. The client can get a copy of the document only. In other words, the contents of the file are determined when the file is created, not when it is used. Of course, the contents in the server can be changed, but the user cannot change them. When a client accesses the document, a copy of the document is sent. The user can then use a browser to see the document. Static documents are prepared using one of several languages: *HyperText Markup Language* (HTML), *Extensible Markup Language* (XML), *Extensible Style Language* (XSL), and *Extensible Hypertext Markup Language* (XHTML).

Dynamic Documents

A **dynamic document** is created by a web server whenever a browser requests the document. When a request arrives, the web server runs an application program or a script that creates the dynamic document. The server returns the result of the program or script as a response to the browser that requested the document. Because a fresh document is created for each request, the contents of a dynamic document may vary from one request to another. A very simple example of a dynamic document is the retrieval of the time and date from a server. Time and date are kinds of information that are dynamic in that they change from moment to moment. The client can ask the server to run a program such as the *date* program in UNIX and send the result of the program to the client. Although the *Common Gateway Interface (CGI)* was used to retrieve a dynamic document in the past, today's options include one of the scripting languages such as *Java Server Pages (JSP)*, which uses the Java language for scripting, or *Active Server Pages (ASP)*, a Microsoft product that uses Visual Basic language for scripting, or *ColdFusion*, which embeds queries in a Structured Query Language (SQL) database in the HTML document.

Active Documents

For many applications, we need a program or a script to be run at the client site. These are called ***active documents***. For example, suppose we want to run a program that creates animated graphics on the screen or a program that interacts with the user. The program definitely needs to be run at the client site where the animation or interaction takes place. When a browser requests an active document, the server sends a copy of the document or a script. The document is then run at the client (browser) site. One way to create an active document is to use *Java applets*, a program written in Java on the server. It is compiled and ready to be run. The document is in bytecode (binary) format. Another way is to use *JavaScripts* but download and run the script at the client site.

HyperText Transfer Protocol (HTTP)

The **HyperText Transfer Protocol (HTTP)** is used to define how the client-server programs can be written to retrieve web pages from the Web. An HTTP client sends a request; an HTTP server returns a response. The server uses the port number 80; the client uses a temporary port number. HTTP uses the services of TCP, which, as discussed before, is a connection-oriented and reliable protocol. This means that, before any transaction between the client and the server can take place, a connection needs to be established between them. After the transaction, the connection should be terminated. The client and server, however, do not need to worry about errors in messages exchanged or loss of any message, because the TCP is reliable and will take care of this

Nonpersistent versus Persistent Connections

Nonpersistent Connections

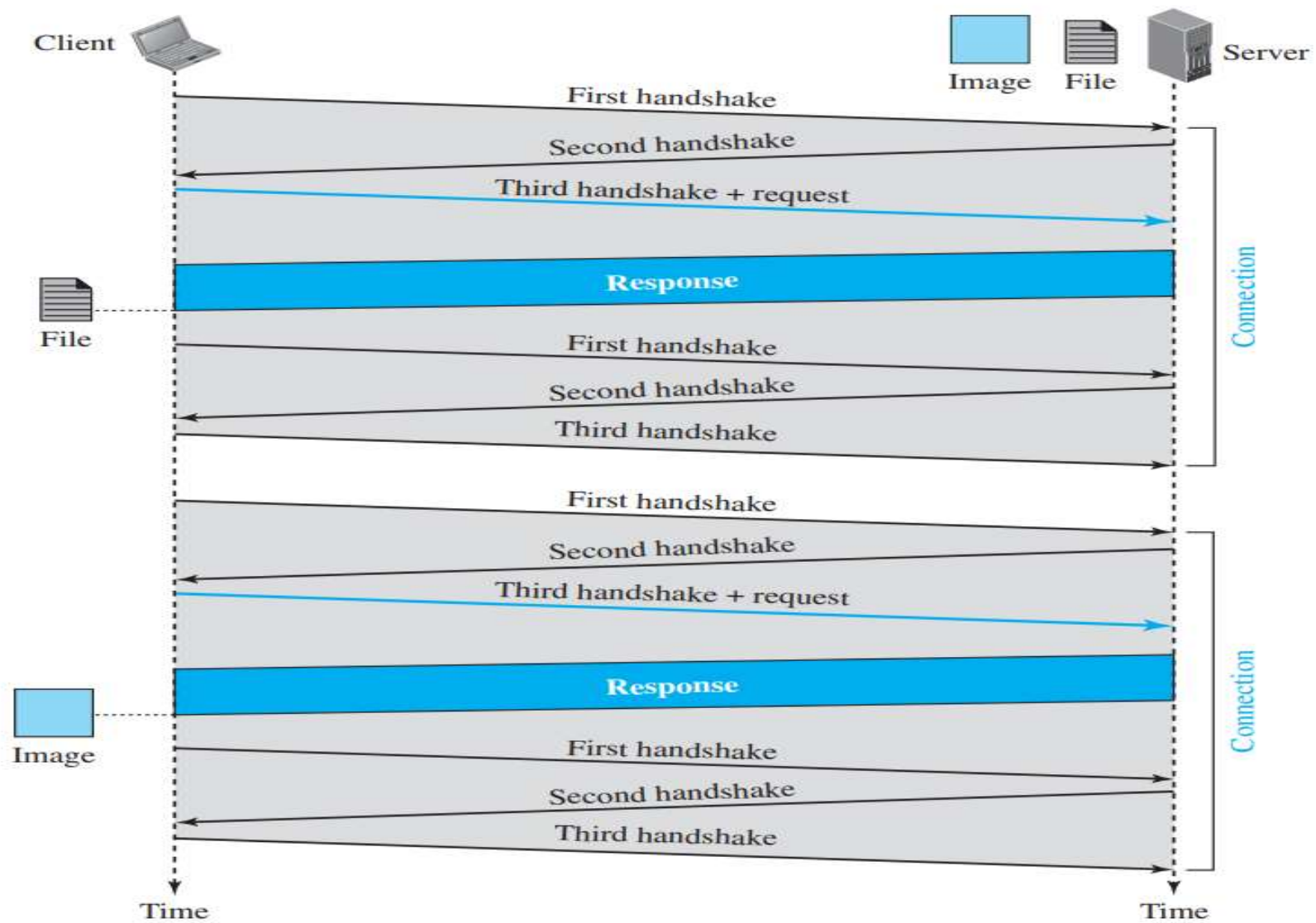
In a **nonpersistent connection**, one TCP connection is made for each request/response. The following lists the steps in this strategy:

1. The client opens a TCP connection and sends a request.
2. The server sends the response and closes the connection.
3. The client reads the data until it encounters an end-of-file marker; it then closes the connection.

In this strategy, if a file contains links to N different pictures in different files (all located on the same server), the connection must be opened and closed $N + 1$ times. The nonpersistent strategy imposes high overhead on the server because the server needs $N + 1$ different buffers each time a connection is opened.

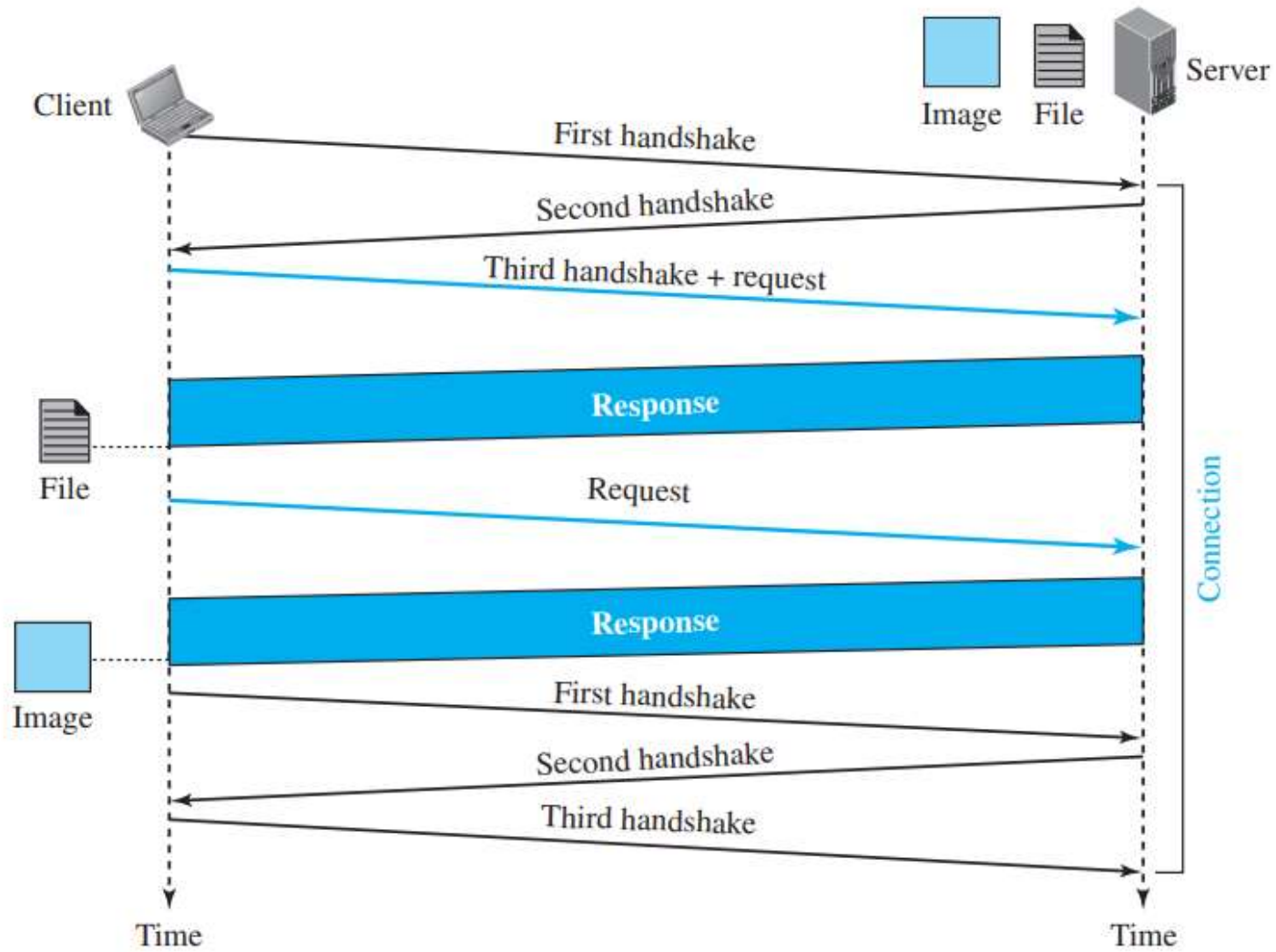
Example of a nonpersistent connection

- The client needs to access a file that contains one link to an image.
- The text file and image are located on the same server. Here we need two connections.
- For each connection, TCP requires at least three handshake messages to establish the connection, but the request can be sent with the third one.
- After the connection is established, the object can be transferred.
- After receiving an object, another three handshake messages are needed to terminate the connection.
- This means that the client and server are involved in two connection establishments and two connection terminations.
- If the transaction involves retrieving 10 or 20 objects, the round trip times spent for these handshakes add up to a big overhead. For each connection the client and server need to allocate extra resources such as buffers and variables.
- This is another burden on both sites, but especially on the server site.



Persistent Connections

- **HTTP version 1.1** specifies a persistent connection by default.
- In a persistent connection, the server leaves the connection open for more requests after sending a response.
- The server can close the connection at the request of a client or if a time-out has been reached.
- The sender usually sends the length of the data with each response. However, there are some occasions when the sender does not know the length of the data.
- This is the case when a document is created dynamically or actively.
- In these cases, the server informs the client that the length is not known and closes the connection after sending the data so the client knows that the end of the data has been reached.
- **Time and resources are saved using persistent connections.**
- Only one set of buffers and variables needs to be set for the connection at each site. The round trip time for connection establishment and connection termination is saved.



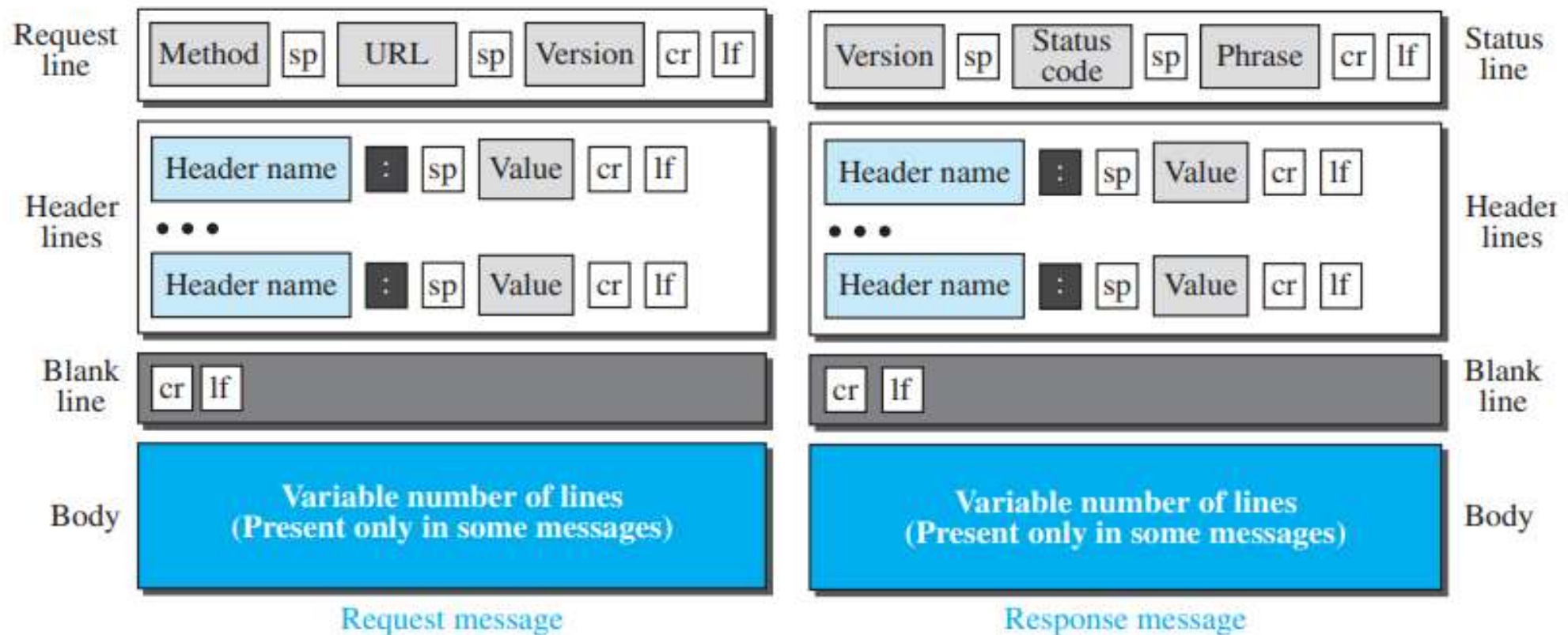
Message Formats

- Each message is made of four sections.
- The first section in the request message is called the request line; the first section in the response message is called the status line.
- The other three sections have the same names in the request and response messages.

Formats of the request and response messages

Legend

sp: Space cr: Carriage Return lf: Line Feed



Request Message

- ✓ The first line in a request message is called a request line.
- ✓ There are three fields in this line separated by one space and terminated by two characters (carriage return and line feed).
- ✓ The fields are called **method**, **URL**, and **version**.

The **method** field defines the request types. In version 1.1 of HTTP, several methods are defined.

- ✓ Most of the time, the client uses the GET method to send a request.
 - In this case, the body of the message is empty.
 - The HEAD method is used when the client needs only some information about the web page from the server, such as the last time it was modified. It can also be used to test the validity of a URL.
 - The response message in this case has only the header section; the body section is empty.
- ✓ The PUT method is the inverse of the GET method; it allows the client to post a new web page on the server (if permitted).
- ✓ The POST method is similar to the PUT method, but it is used to send some information to the server to be added to the web page or to modify the web page.
- ✓ The TRACE method is used for debugging; the client asks the server to echo back the request to check whether the server is getting the requests.
- ✓ The DELETE method allows the client to delete a web page on the server if the client has permission to do so.
- ✓ The CONNECT method was originally made as a reserve method; it may be used by proxy servers.
- ✓ Finally, the OPTIONS method allows the client to ask about the properties of a web page.

<i>Method</i>	<i>Action</i>
GET	Requests a document from the server
HEAD	Requests information about a document but not the document itself
PUT	Sends a document from the client to the server
POST	Sends some information from the client to the server
TRACE	Echoes the incoming request
DELETE	Removes the web page
CONNECT	Reserved
OPTIONS	Inquires about available options

- The second field is **URL**. It defines the address and name of the corresponding web page.
- The third field, **version**, gives the version of the protocol; the most current version of HTTP is 1.1.

- After the request line, we can have zero or more **request header lines**.
 - Each header line sends additional information from the client to the server. For example, the client can request that the document be sent in a special format.
 - Each header line has a header name, a colon, a space, and a header value.
 - The value field defines the values associated with each header name.
 - The list of values can be found in the corresponding RFCs.
-
- The body can be present in a request message.
 - Usually, it contains the content to be sent or the file to be published on the website when the method is PUT or POST.

Request header names

<i>Header</i>	<i>Description</i>
User-agent	Identifies the client program
Accept	Shows the media format the client can accept
Accept-charset	Shows the character set the client can handle
Accept-encoding	Shows the encoding scheme the client can handle
Accept-language	Shows the language the client can accept
Authorization	Shows what permissions the client has
Host	Shows the host and port number of the client
Date	Shows the current date
Upgrade	Specifies the preferred communication protocol
Cookie	Returns the cookie to the server (explained later)
If-Modified-Since	If the file is modified since a specific date

Response Message

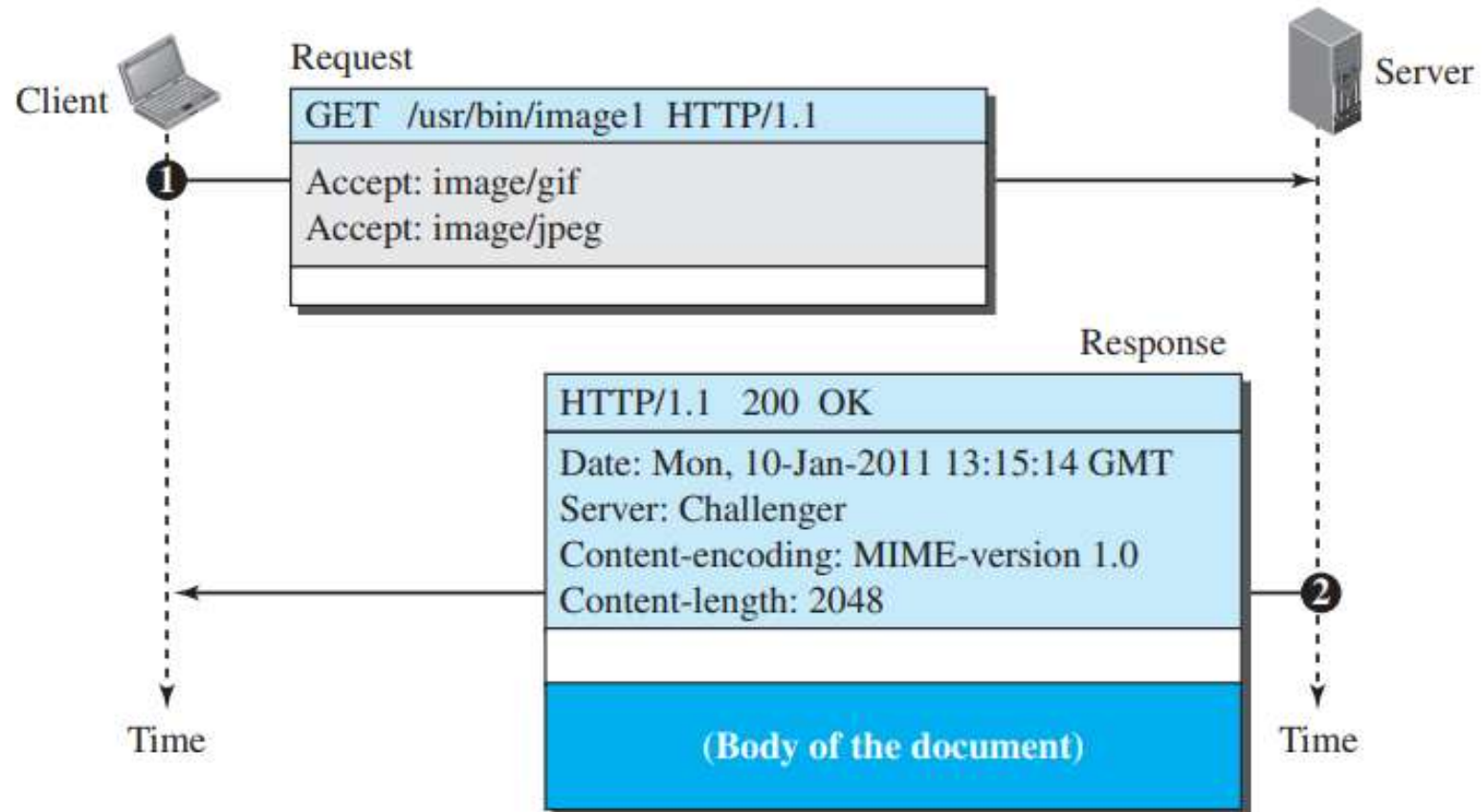
- A response message consists of a status line, header lines, a blank line, and sometimes a body.
- The first line in a response message is called the status line.
- There are three fields in this line separated by spaces and terminated by a carriage return and line feed.
- The first field defines the version of HTTP protocol, currently 1.1.
 - The status code field defines the status of the request. It consists of three digits. Whereas the codes in the 100 range are only informational, the codes in the 200 range indicate a successful request. The codes in the 300 range redirect the client to another URL, and the codes in the 400 range indicate an error at the client site. Finally, the codes in the 500 range indicate an error at the server site. The status phrase explains the status code in text form.
- After the status line, we can have zero or more response header lines.
 - Each header line sends additional information from the server to the client. For example, the sender can send extra information about the document.
 - Each header line has a header name, a colon, a space, and a header value.

Response header names

<i>Header</i>	<i>Description</i>
Date	Shows the current date
Upgrade	Specifies the preferred communication protocol
Server	Gives information about the server
Set-Cookie	The server asks the client to save a cookie
Content-Encoding	Specifies the encoding scheme
Content-Language	Specifies the language
Content-Length	Shows the length of the document
Content-Type	Specifies the media type
Location	To ask the client to send the request to another site
Accept-Ranges	The server will accept the requested byte-ranges
Last-modified	Gives the date and time of the last change

- The body contains the document to be sent from the server to the client. The body is present unless the response is an error message.

Document retrieval process



Cookies

- Websites are being used as electronic stores that allow users to browse through the store, select wanted items, put them in an electronic cart, and pay at the end with a credit card.
- Some websites need to allow access to registered clients only.
- Some websites are used as portals: the user selects the web pages he wants to see.
- Some websites are just advertising agencies.

For these purposes, the **cookie** mechanism was devised.

Creating and Storing Cookies

The creation and storing of cookies depend on the implementation; however, the principle is the same.

1. When a server receives a request from a client, it stores information about the client in a file or a string. The information may include the domain name of the client, the contents of the cookie (information the server has gathered about the client such as name, registration number, and so on), a timestamp, and other information depending on the implementation.
2. The server includes the cookie in the response that it sends to the client.
3. When the client receives the response, the browser stores the cookie in the cookie directory, which is sorted by the server domain name.

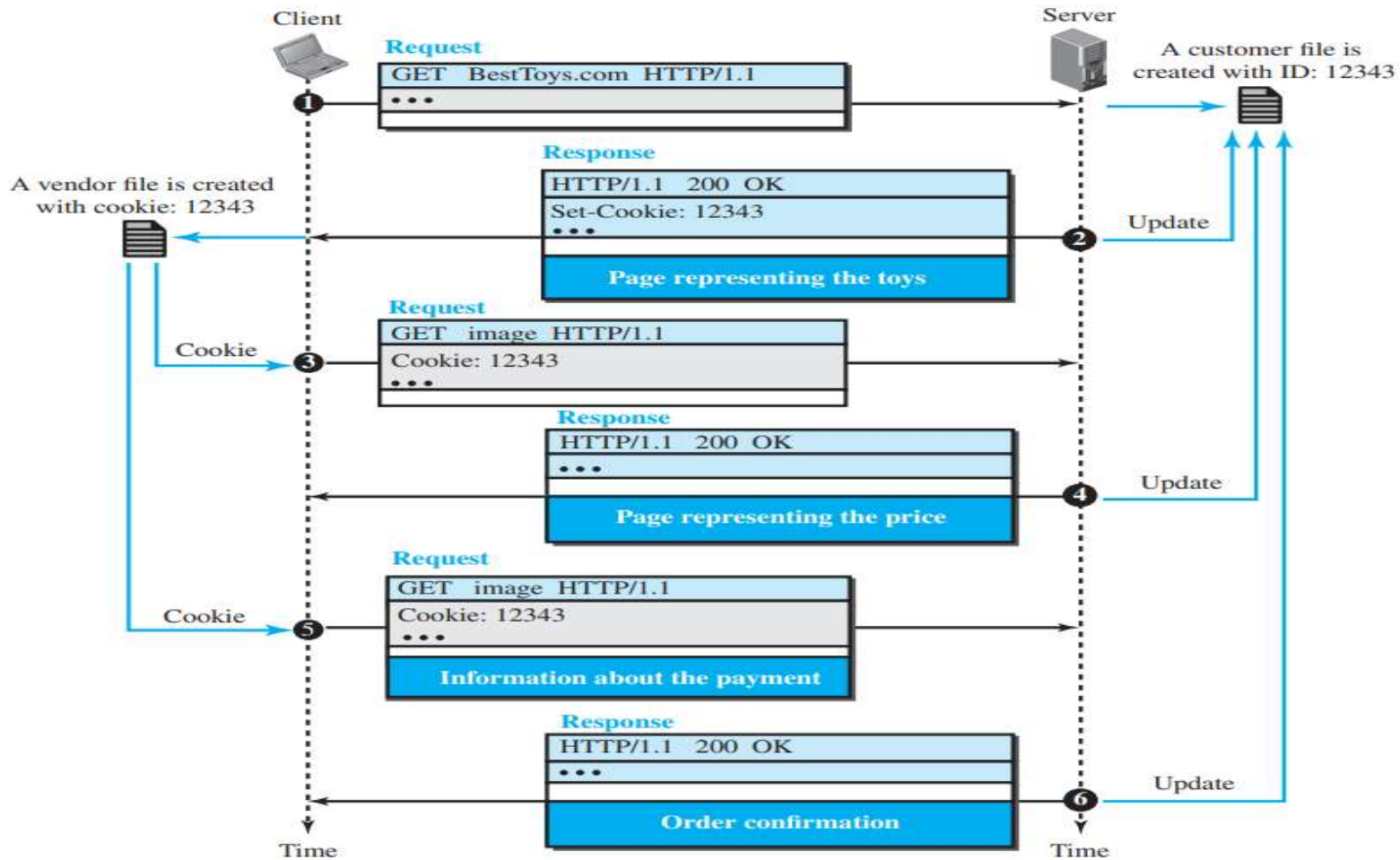
Using Cookies

When a client sends a request to a server, the browser looks in the cookie directory to see if it can find a cookie sent by that server. If found, the cookie is included in the request. When the server receives the request, it knows that this is an old client, not a new one. Note that the contents of the cookie are never read by the browser or disclosed to the user. It is a cookie *made* by the server and *eaten* by the server. Now let us see how a cookie is used for the four previously mentioned purposes:

- ❑ An *electronic store* (e-commerce) can use a cookie for its client shoppers. When a client selects an item and inserts it in a cart, a cookie that contains information about the item, such as its number and unit price, is sent to the browser. If the client selects a second item, the cookie is updated with the new selection information, and so on. When the client finishes shopping and wants to check out, the last cookie is retrieved and the total charge is calculated.

- ❑ The site that restricts access to *registered clients* only sends a cookie to the client when the client registers for the first time. For any repeated access, only those clients that send the appropriate cookie are allowed.
- ❑ A web *portal* uses the cookie in a similar way. When a user selects her favorite pages, a cookie is made and sent. If the site is accessed again, the cookie is sent to the server to show what the client is looking for.
- ❑ A cookie is also used by *advertising* agencies. An advertising agency can place banner ads on some main website that is often visited by users. The advertising agency supplies only a URL that gives the advertising agency's address instead of the banner itself. When a user visits the main website and clicks the icon of a corporation, a request is sent to the advertising agency. The advertising agency sends the requested banner, but it also includes a cookie with the ID of the user. Any future use of the banners adds to the database that profiles the Web behavior of the user. The advertising agency has compiled the interests of the user and can sell this information to other parties. This use of cookies has made them very controversial. Hopefully, some new regulations will be devised to preserve the privacy of users.

A scenario in which an electronic store can benefit from the use of cookies



Web Caching: Proxy Servers

HTTP supports **proxy servers**. A proxy server is a computer that keeps copies of responses to recent requests. The HTTP client sends a request to the proxy server. The proxy server checks its cache. If the response is not stored in the cache, the proxy server sends the request to the corresponding server. Incoming responses are sent to the proxy server and stored for future requests from other clients.

The proxy server reduces the load on the original server, decreases traffic, and improves latency. However, to use the proxy server, the client must be configured to access the proxy instead of the target server.

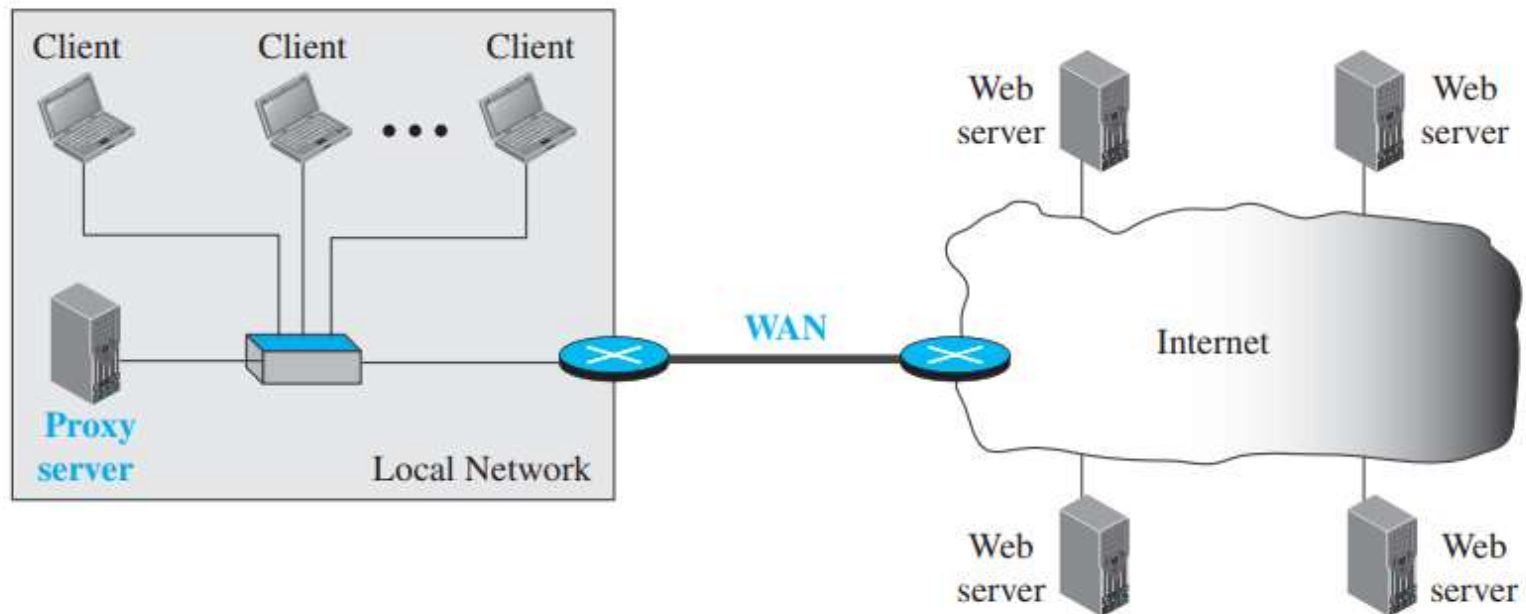
Note that the proxy server acts as both server and client. When it receives a request from a client for which it has a response, it acts as a server and sends the response to the client. When it receives a request from a client for which it does not have a response, it first acts as a client and sends a request to the target server. When the response has been received, it acts again as a server and sends the response to the client.

Proxy Server Location

The proxy servers are normally located at the client site. This means that we can have a hierarchy of proxy servers, as shown below:

1. A client computer can also be used as a proxy server, in a small capacity, that stores responses to requests often invoked by the client.
2. In a company, a proxy server may be installed on the computer LAN to reduce the load going out of and coming into the LAN.
3. An ISP with many customers can install a proxy server to reduce the load going out of and coming into the ISP network.

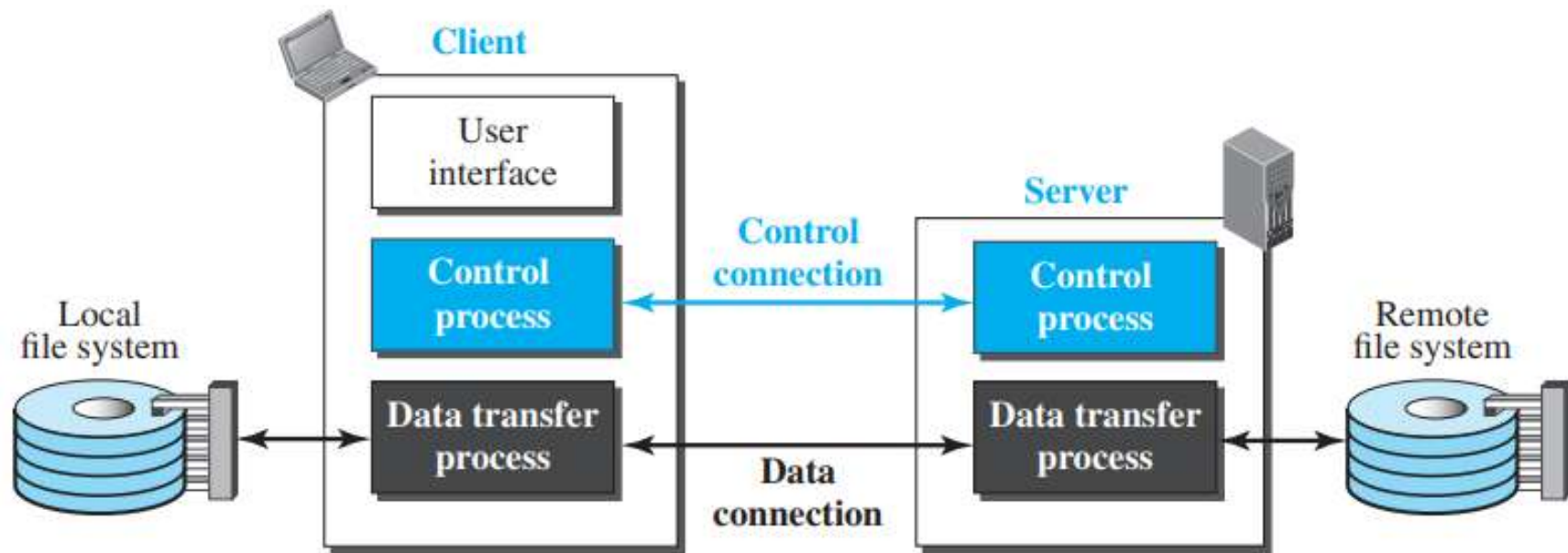
Example of a proxy server



FTP

File Transfer Protocol

- File Transfer Protocol (FTP) is the standard protocol provided by TCP/IP for copying a file from one host to another. Although transferring files from one system to another seems simple and straightforward, some problems must be dealt with first.
- For example, two systems may use different file name conventions. Two systems may have different ways to represent data. Two systems may have different directory structures. All of these problems have been solved by FTP in a very simple and elegant approach. Although we can transfer files using HTTP, FTP is a better choice to transfer large files or to transfer files using different formats.



- The **client has three components**: the user interface, the client control process, and the client data transfer process.
- The **server has two components**: the server control process and the server data transfer process.
- The **control connection** is made between the control processes.
- The **data connection** is made between the data transfer processes.
- **Separation of commands and data transfer** makes FTP more efficient. The control connection uses very simple rules of communication. We need to transfer only a line of command or a line of response at a time. The data connection, on the other hand, needs more complex rules due to the variety of data types transferred.

Two Connections

The two connections in FTP have different lifetimes. The control connection remains connected during the entire interactive FTP session. The data connection is opened and then closed for each file transfer activity. It opens each time commands that involve transferring files are used, and it closes when the file is transferred. In other words, when a user starts an FTP session, the control connection opens. While the control connection is open, the data connection can be opened and closed multiple times if several files are transferred. FTP uses two well-known TCP ports: port 21 is used for the control connection, and port 20 is used for the data connection.

Control Connection

For control communication, FTP uses the same approach as TELNET. It uses the NVT ASCII character set as used by TELNET. Communication is achieved through commands and responses. This simple method is adequate for the control connection because we send one command (or response) at a time. Each line is terminated with a two-character (carriage return and line feed) end-of-line token.

During this control connection, commands are sent from the client to the server and responses are sent from the server to the client. Commands, which are sent from the FTP client control process, are in the form of ASCII uppercase, which may or may not be followed by an argument.

Some FTP commands

<i>Command</i>	<i>Argument(s)</i>	<i>Description</i>
ABOR		Abort the previous command
CDUP		Change to parent directory
CWD	Directory name	Change to another directory
DELE	File name	Delete a file
LIST	Directory name	List subdirectories or files
MKD	Directory name	Create a new directory
PASS	User password	Password
PASV		Server chooses a port
PORT	Port identifier	Client chooses a port
PWD		Display name of current directory
QUIT		Log out of the system
RETR	File name(s)	Retrieve files; files are transferred from server to client
RMD	Directory name	Delete a directory
RNFR	File name (old)	Identify a file to be renamed
RNTO	File name (new)	Rename the file
STOR	File name(s)	Store files; file(s) are transferred from client to server
STRU	F, R, or P	Define data organization (F : file, R : record, or P : page)
TYPE	A, E, I	Default file type (A : ASCII, E : EBCDIC, I : image)
USER	User ID	User information
MODE	S, B, or C	Define transmission mode (S : stream, B : block, or C : compressed)

Some responses in FTP

<i>Code</i>	<i>Description</i>	<i>Code</i>	<i>Description</i>
125	Data connection open	250	Request file action OK
150	File status OK	331	User name OK; password is needed
200	Command OK	425	Cannot open data connection
220	Service ready	450	File action not taken; file not available
221	Service closing	452	Action aborted; insufficient storage
225	Data connection open	500	Syntax error; unrecognized command
226	Closing data connection	501	Syntax error in parameters or arguments
230	User login OK	530	User not logged in

Data Connection

The data connection uses the well-known port 20 at the server site. However, the creation of a data connection is different from the control connection. The following shows the steps:

1. The client, not the server, issues a passive open using an ephemeral port. This must be done by the client because it is the client that issues the commands for transferring files.
2. Using the PORT command the client sends this port number to the server.
3. The server receives the port number and issues an active open using the well-known port 20 and the received ephemeral port number.

Communication over Data Connection

The purpose and implementation of the data connection are different from those of the control connection. We want to transfer files through the data connection. The client must define the type of file to be transferred, the structure of the data, and the transmission mode. Before sending the file through the data connection, we prepare for transmission through the control connection. The heterogeneity problem is resolved by defining three attributes of communication: file type, data structure, and transmission mode.

File Type

FTP can transfer one of the following file types across the data connection: ASCII file, EBCDIC file, or image file.

Data Structure

FTP can transfer a file across the data connection using one of the following interpretations of the structure of the data: *file structure*, *record structure*, or *page structure*. The file structure format (used by default) has no structure. It is a continuous stream of bytes. In the record structure, the file is divided into *records*. This can be used only with text files. In the page structure, the file is divided into pages, with each page having a page number and a page header. The pages can be stored and accessed randomly or sequentially.

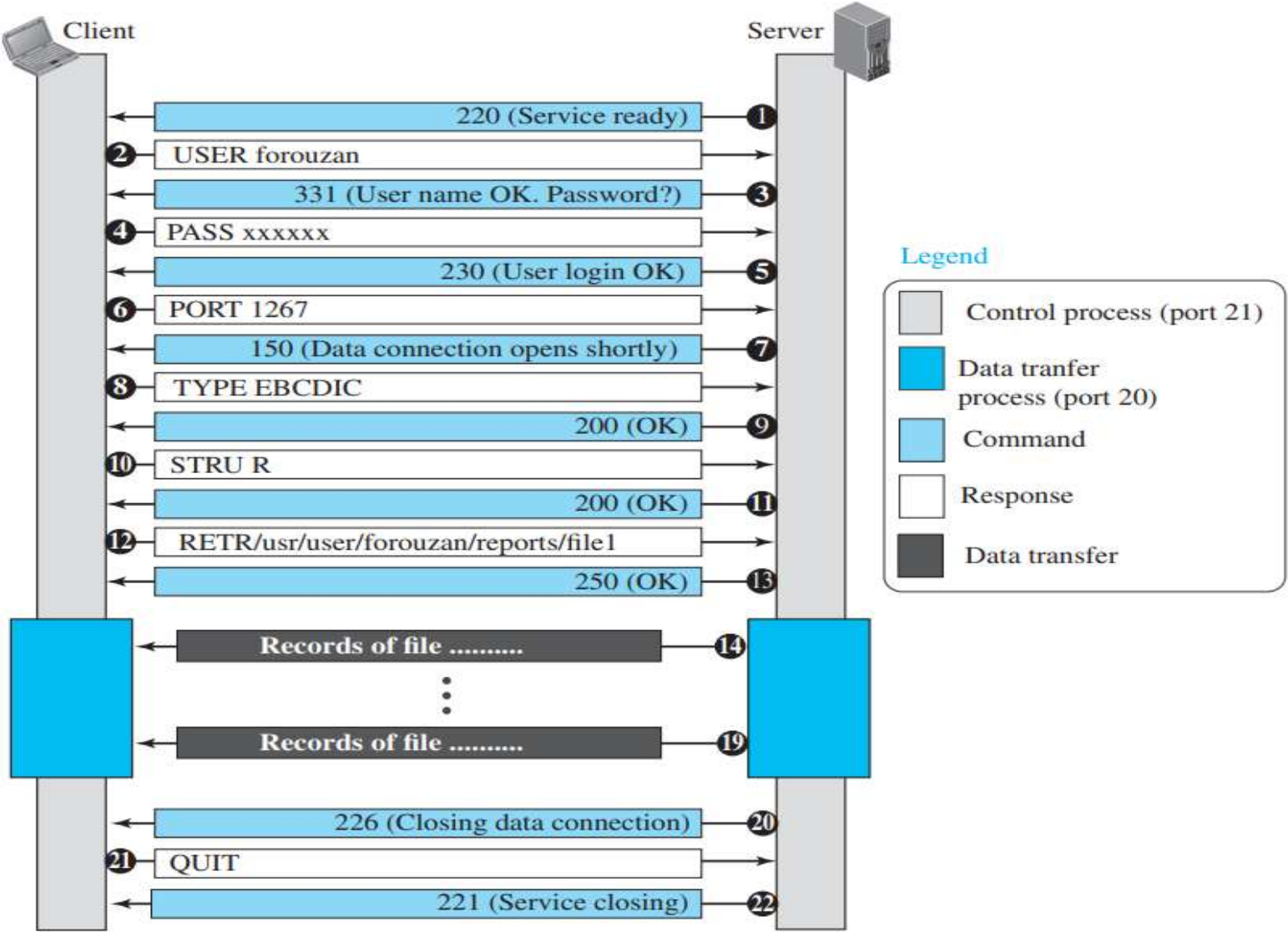
Transmission Mode

FTP can transfer a file across the data connection using one of the following three transmission modes: *stream mode*, *block mode*, or *compressed mode*. The stream mode is the default mode; data are delivered from FTP to TCP as a continuous stream of bytes. In the block mode, data can be delivered from FTP to TCP in blocks. In this case, each block is preceded by a 3-byte header. The first byte is called the *block descriptor*; the next two bytes define the size of the block in bytes.

File Transfer

File transfer occurs over the data connection under the control of the commands sent over the control connection. However, we should remember that file transfer in FTP means one of three things: *retrieving a file* (server to client), *storing a file* (client to server), and *directory listing* (server to client).

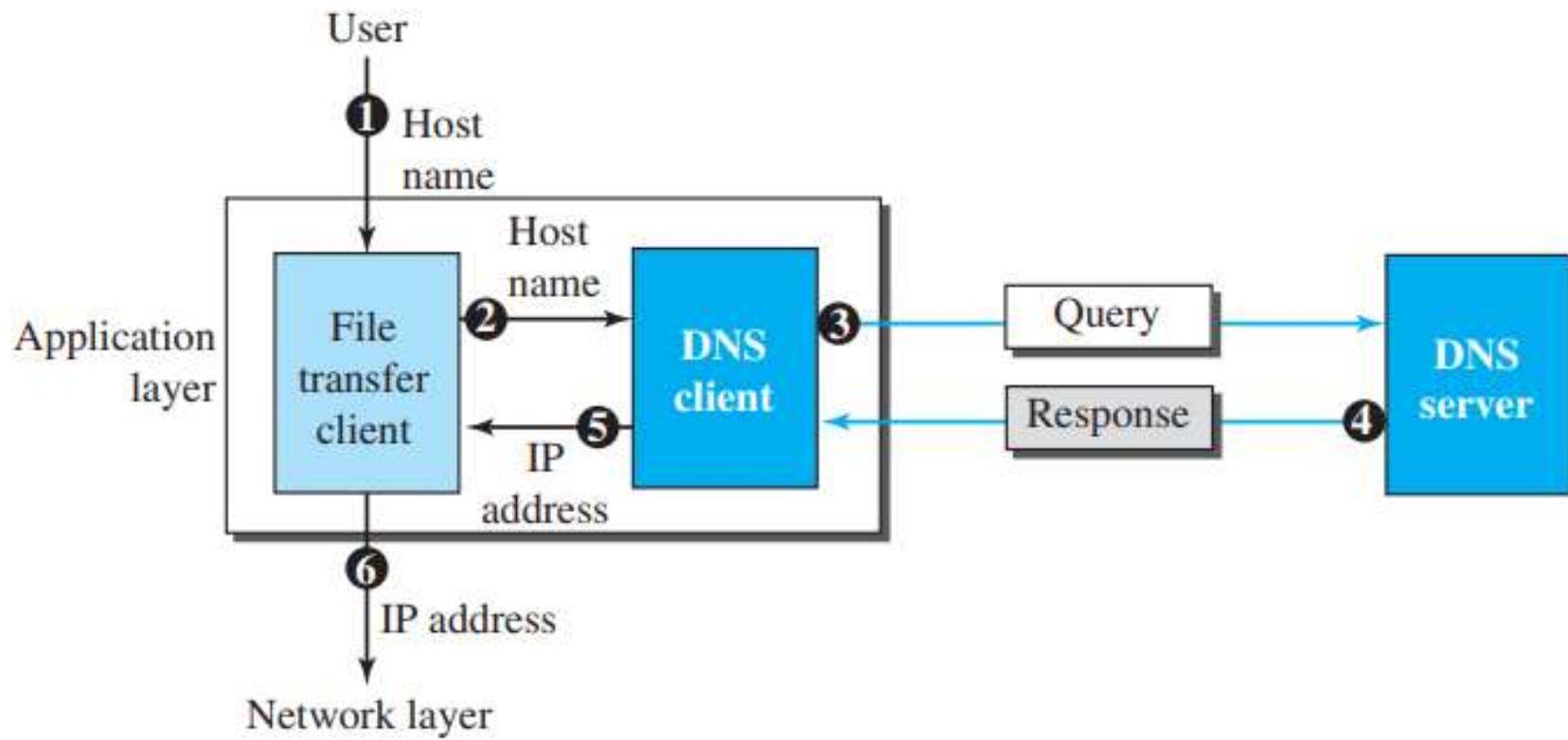
FTP for retrieving a file



DOMAIN NAME SYSTEM (DNS)

- To identify an entity, TCP/IP protocols use the IP address, which uniquely identifies the connection of a host to the Internet.
- However, people prefer to use names instead of numeric addresses.
- Therefore, the Internet needs to have a directory system that can map a name to an address.
- A better solution is to distribute the information among many computers in the world.
- In this method, the host that needs mapping can contact the closest computer holding the needed information.
- This method is used by the **Domain Name System (DNS)**.

Purpose of DNS



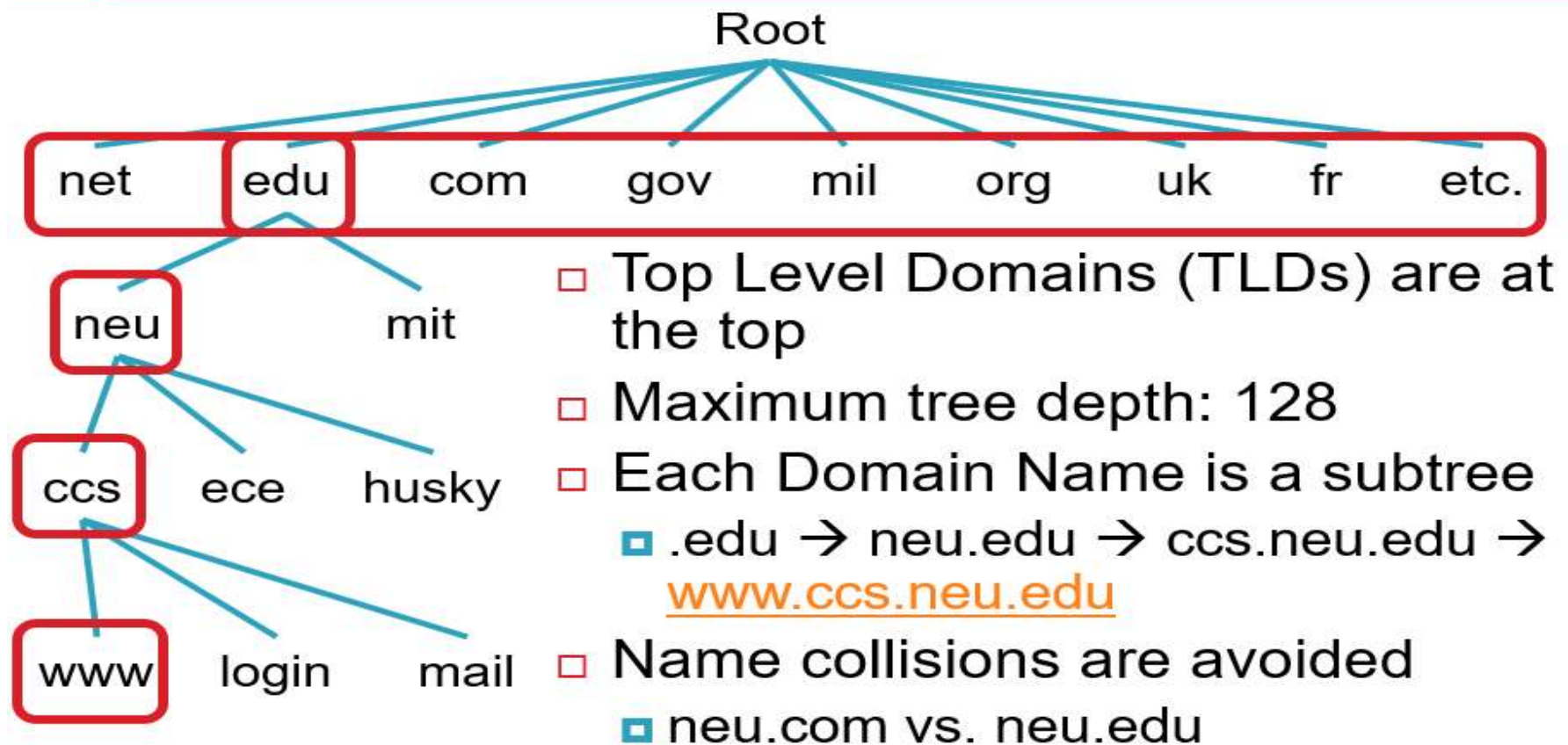
A user wants to use a file transfer client to access the corresponding file transfer server running on a remote host. The user knows only the file transfer server name, such as `afilesource.com`. However, the TCP/IP suite needs the IP address of the file transfer server to make the connection.

The following **six steps map the host name to an IP address**:

1. The user passes the host name to the file transfer client.
2. The file transfer client passes the host name to the DNS client.
3. Each computer, after being booted, knows the address of one DNS server. The DNS client sends a message to a DNS server with a query that gives the file transfer server name using the known IP address of the DNS server.
4. The DNS server responds with the IP address of the desired file transfer server.
5. The DNS server passes the IP address to the file transfer client.
6. The file transfer client now uses the received IP address to access the file transfer server.

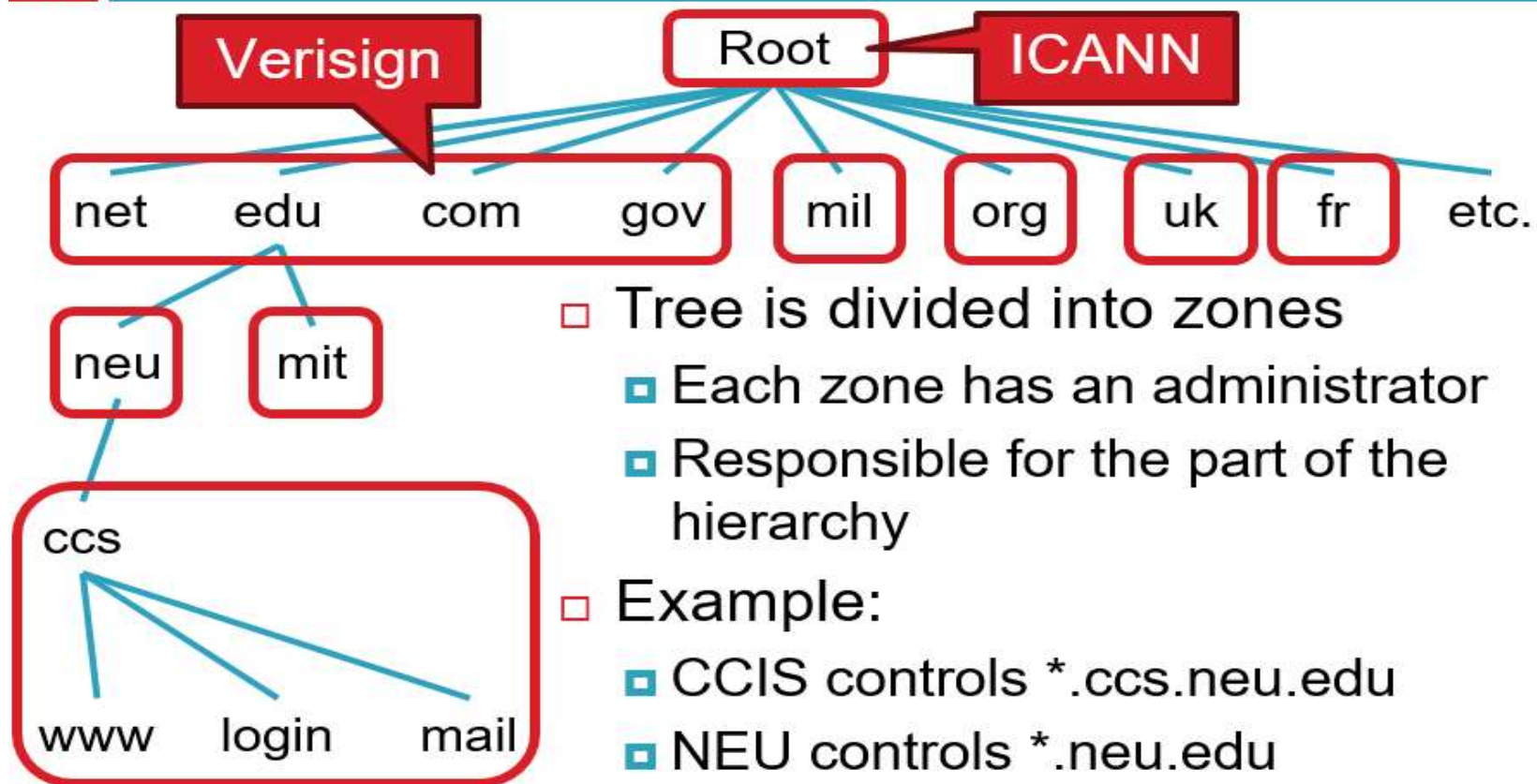
Naming Hierarchy

9



Hierarchical Administration

10



Server Hierarchy

11

- ❑ Functions of each DNS server:
 - ❑ Authority over a portion of the hierarchy
 - No need to store all DNS names
 - ❑ Store all the records for hosts/domains in its zone
 - May be replicated for robustness
 - ❑ Know the addresses of the root servers
 - Resolve queries for unknown names
- ❑ Root servers know about all TLDs
 - ❑ The buck stops at the root servers

Root Name Servers

12

- ❑ Responsible for the Root Zone File

- ❑ Lists the TLDs and who controls them
- ❑ ~272KB in size

com.	172800	IN	NS	a.gtld-servers.net.
com.	172800	IN	NS	b.gtld-servers.net.
com.	172800	IN	NS	c.gtld-servers.net.

- ❑ Administered by ICANN

- ❑ 13 root servers, labeled A→M
- ❑ 6 are anycasted, i.e. they are globally replicated

- ❑ Contacted when names cannot be resolved

- ❑ In practice, most systems cache this information

Local Name Servers

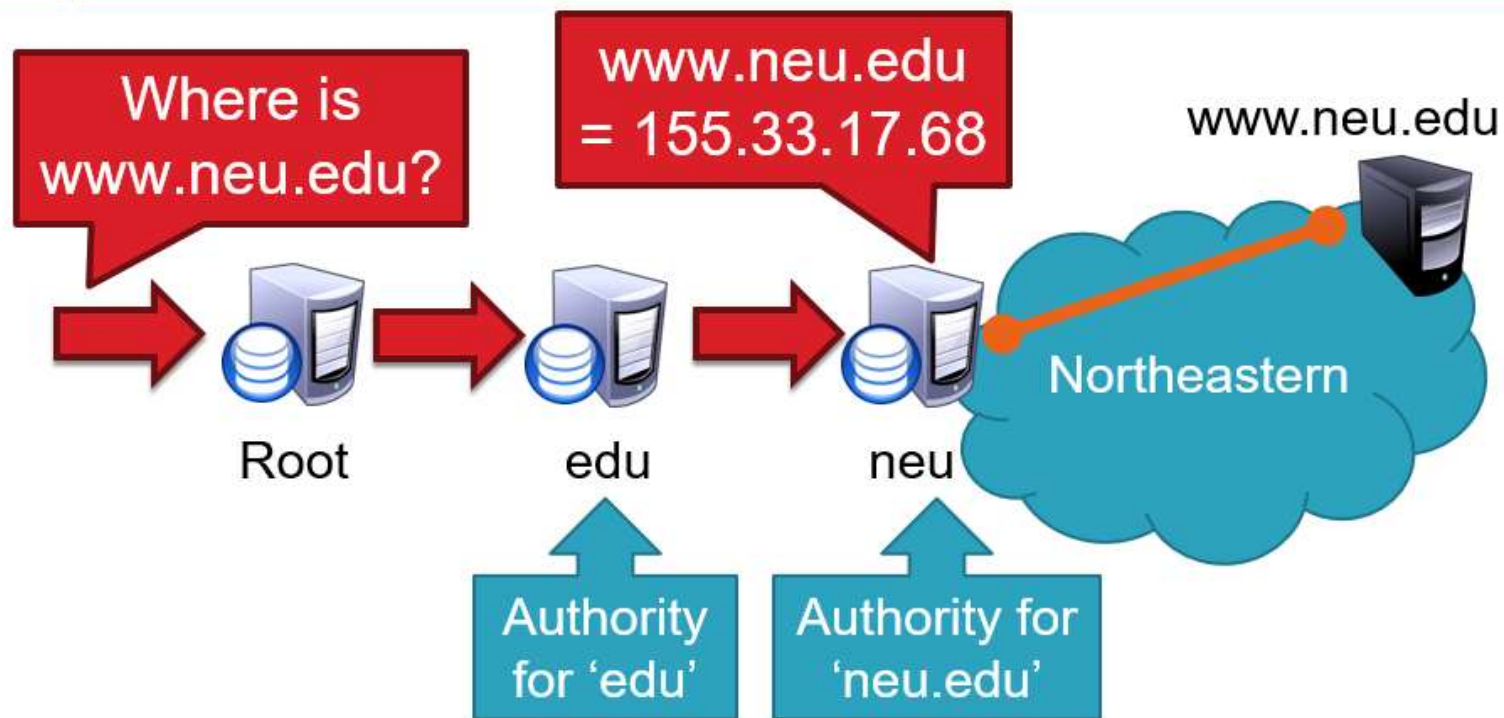
14



- ❑ Each ISP/company has a local, default name server
- ❑ Often configured via DHCP
- ❑ Hosts begin DNS queries by contacting the local name server
- ❑ Frequently cache query results

Authoritative Name Servers

15



- Stores the name → IP mapping for a given host

Basic Domain Name Resolution

16

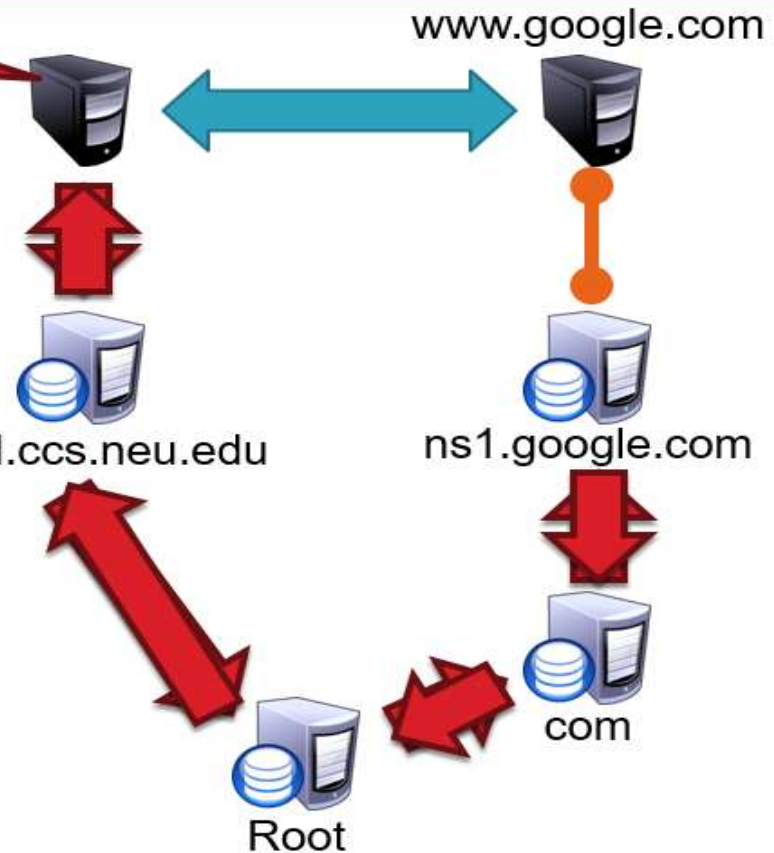
- Every host knows a local DNS server
 - ▣ Sends all queries to the local DNS server
- If the local DNS can answer the query, then you're done
 1. Local server is also the authoritative server for that name
 2. Local server has cached the record for that name
- Otherwise, go down the hierarchy and search for the authoritative name server
 - ▣ Every local DNS server knows the root servers
 - ▣ Use cache to skip steps if possible

Recursive DNS Query

17

Where is www.google.com?

- Puts the burden of resolution on the contacted name server
- How does asgard know who to forward responses too?
 - asgard.ccs.neu.edu
 - Random IDs embedded in DNS queries

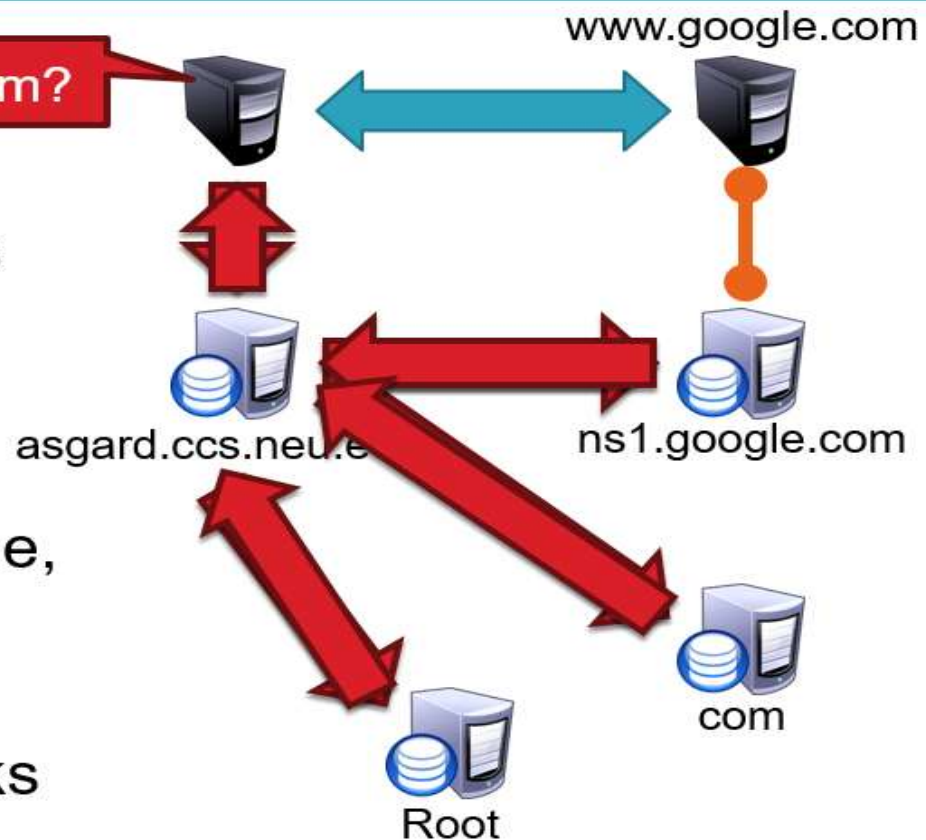


Iterated DNS query

18

Where is www.google.com?

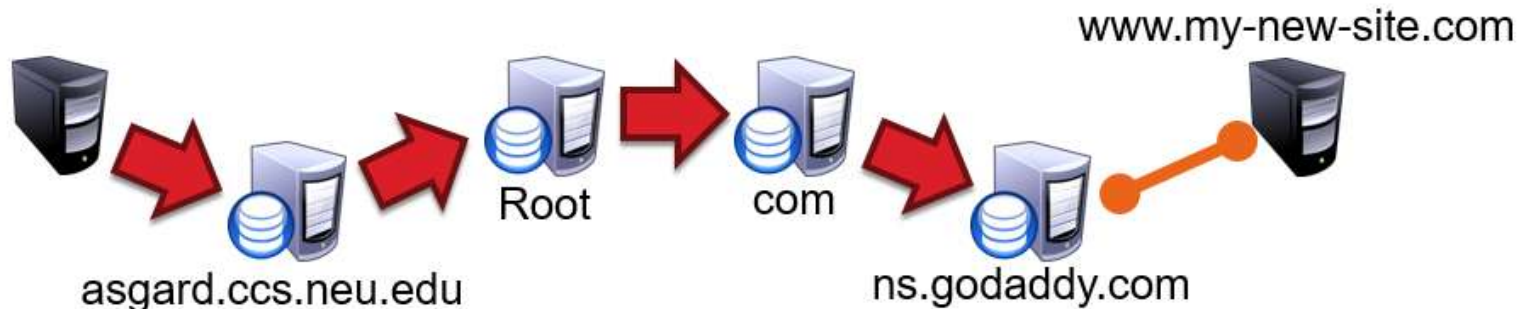
- Contact server replies with the name of the next authority in the hierarchy
- “I don’t know this name, but this other server might”
- This is how DNS works today



DNS Propagation

19

- ❑ How many of you have purchased a domain name?
 - ▣ Did you notice that it took ~72 hours for your name to become accessible?
 - ▣ This delay is called DNS Propagation

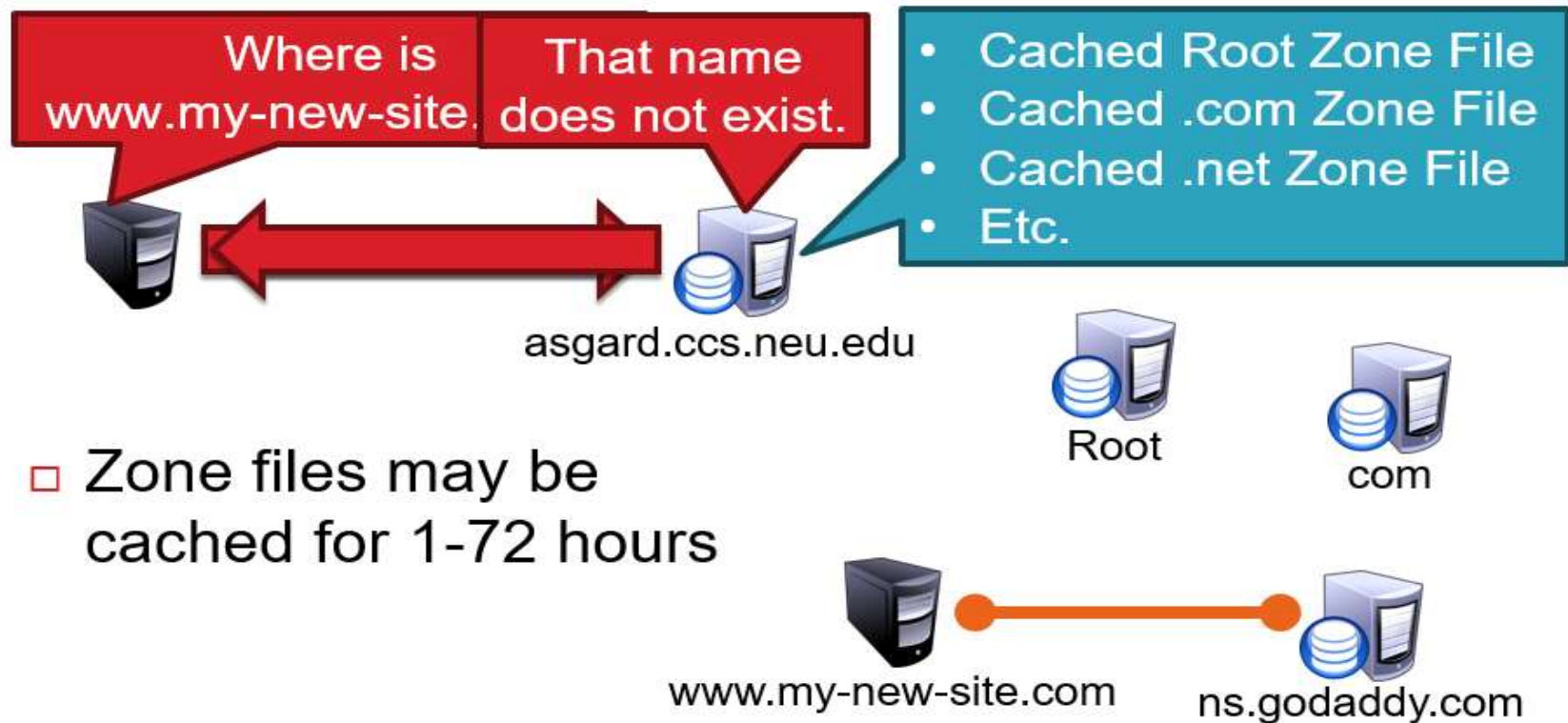


- ❑ Why would this process fail for a new DNS name?

Caching vs. Freshness

20

- DNS Propagation delay is caused by caching



- Zone files may be cached for 1-72 hours

DNS Resource Records

21

- ❑ DNS queries have two fields: **name** and **type**
- ❑ Resource record is the response to a query
 - ▣ Four fields: (**name**, **value**, **type**, TTL)
 - ▣ There may be multiple records returned for one query
- ❑ What do the **name** and **value** mean?
 - ▣ Depends on the **type** of query and response

DNS Types

22

- Type = A / AAAA
 - ▣ Name = domain name
 - ▣ Value = IP address
 - ▣ A is IPv4, AAAA is IPv6

- Type = NS
 - ▣ Name = partial domain
 - ▣ Value = name of DNS server for this domain
 - ▣ “Go send your query to this other server”

Query

Name: www.ccs.neu.edu
Type: A

Resp.

Name: www.ccs.neu.edu
Value: 129.10.116.81

Query

Name: ccs.neu.edu
Type: NS

Resp.

Name: ccs.neu.edu
Value: 129.10.116.51

DNS Types, Continued

23

□ Type = CNAME

- ▣ Name = hostname
- ▣ Value = canonical hostname
- ▣ Useful for aliasing
- ▣ CDNs use this

Query

Name: [foo.mysite.com](#)
Type: CNAME

Resp.

Name: [foo.mysite.com](#)
Value: [bar.mysite.com](#)

□ Type = MX

- ▣ Name = domain in email address
- ▣ Value = canonical name of mail server

Query

Name: [ccs.neu.edu](#)
Type: MX

Resp.

Name: [ccs.neu.edu](#)
Value: [amber.ccs.neu.edu](#)

Reverse Lookups

24

- ❑ What about the IP→name mapping?
- ❑ Separate server hierarchy stores reverse mappings
 - ▣ Rooted at in-addr.arpa and ip6.arpa
- ❑ Additional DNS record **type**: PTR
 - ▣ Name = IP address
 - ▣ Value = domain name
- ❑ Not guaranteed to exist for all IPs

Query

Name: 129.10.116.51
Type: PTR

Resp.

Name: 129.10.116.51
Value: ccs.neu.edu

DNS as Indirection Service

25

- DNS gives us very powerful capabilities
 - ▣ Not only easier for humans to reference machines!
- Changing the IPs of machines becomes trivial
 - ▣ e.g. you want to move your web server to a new host
 - ▣ Just change the DNS record!

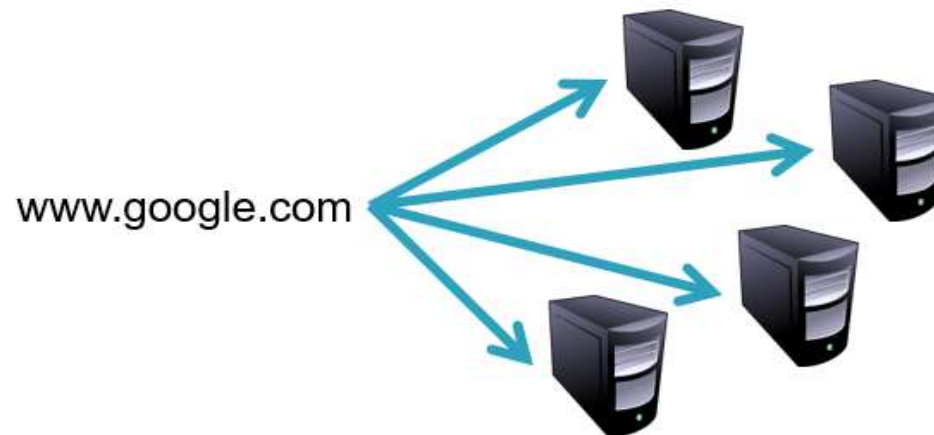
Aliasing and Load Balancing

26

- One machine can have many aliases



- One domain can map to multiple machines



Dynamic Host Configuration Protocol (DHCP)

- DHCP is an application-layer program, using the client-server paradigm, that actually helps TCP/IP at the network layer.
- DHCP has found such widespread use in the Internet that it is often called a **plug-and-play protocol**.
- It can be used in many situations.
- A network manager can configure DHCP to assign permanent IP addresses to the host and routers.
- DHCP can also be configured to provide temporary, on demand, IP addresses to hosts.
- The second capability can provide a temporary IP address to a traveller to connect her laptop to the Internet while she is staying in the hotel. It also allows an ISP with 1000 granted addresses to provide services to 4000 households, assuming not more than one-fourth of customers use the Internet at the same time.
- Four pieces of information are normally needed: **the computer address, the prefix, the address of a router, and the IP address of a name server**. DHCP can be used to provide these pieces of information to the host.

DHCP message format

0	8	16	24	31
Opcode	Htype	HLen	HCount	
Transaction ID				
Time elapsed		Flags		
Client IP address				
Your IP address				
Server IP address				
Gateway IP address				
Client hardware address				
Server name				
Boot file name				
Options				

Fields:

Opcode: Operation code, request (1) or reply (2)

Htype: Hardware type (Ethernet, ...)

HLen: Length of hardware address

HCount: Maximum number of hops the packet can travel

Transaction ID: An integer set by the client and repeated by the server

Time elapsed: The number of seconds since the client started to boot

Flags: First bit defines unicast (0) or multicast (1); other 15 bits not used

Client IP address: Set to 0 if the client does not know it

Your IP address: The client IP address sent by the server

Server IP address: A broadcast IP address if client does not know it

Gateway IP address: The address of default router

Server name: A 64-byte domain name of the server

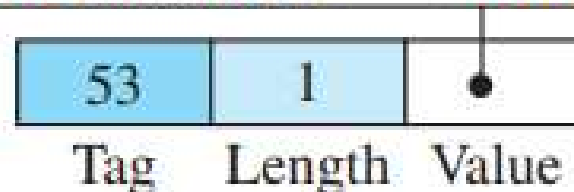
Boot file name: A 128-byte file name holding extra information

Options: A 64-byte field with dual purpose described in text

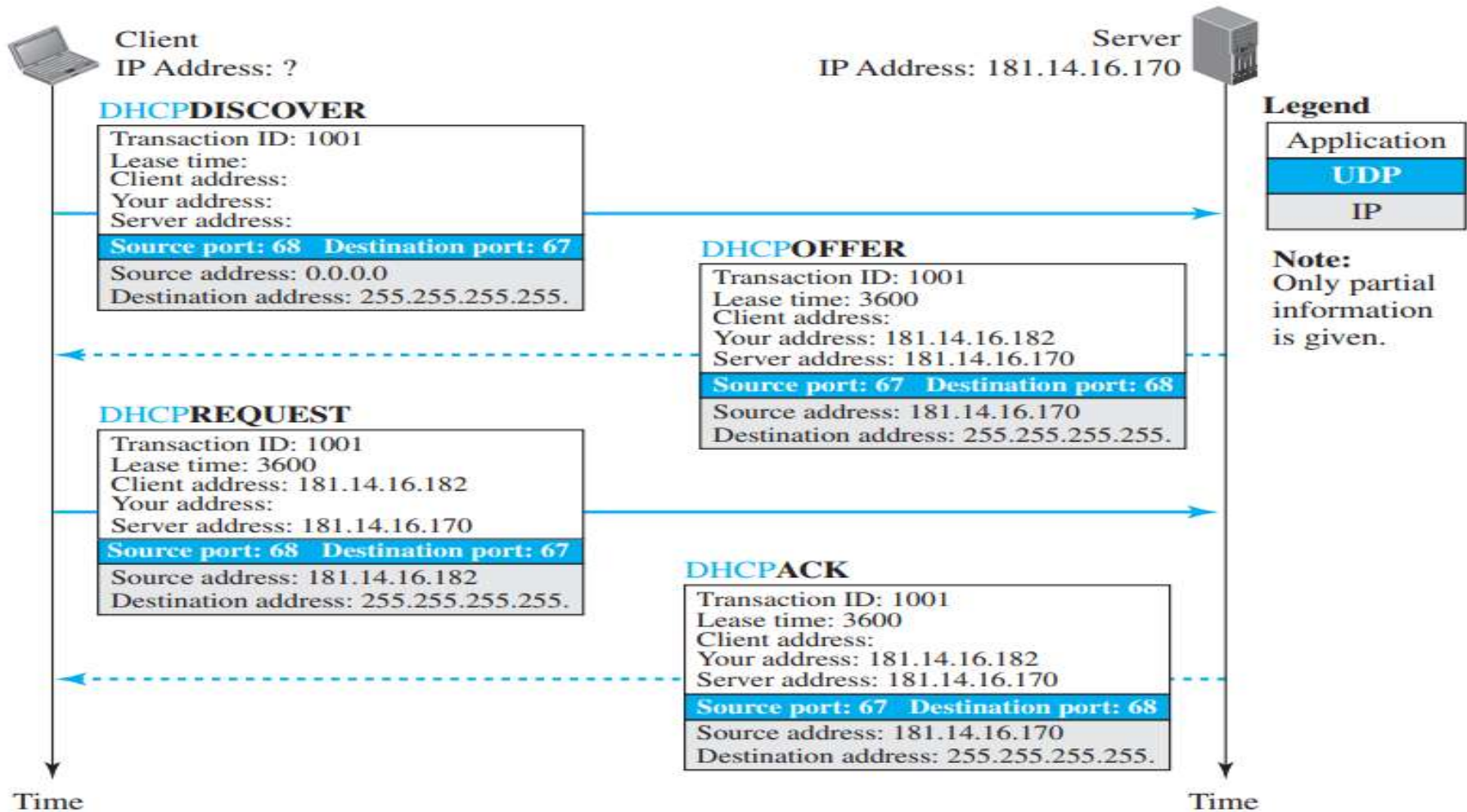
The 64-byte option field has a dual purpose. It can carry either additional information or some specific vendor information. The server uses a number, called a **magic cookie**, in the format of an IP address with the value of 99.130.83.99. When the client finishes reading the message, it looks for this magic cookie. If present, the next 60 bytes are options. An option is composed of three fields: a 1-byte tag field, a 1-byte length field, and a variable-length value field. There are several tag fields that are mostly used by vendors. If the tag field is 53, the value field defines one of the 8 message types

Option format

1	DHCP DISCOVER	5	DHCP ACK
2	DHCP OFFER	6	DHCP NACK
3	DHCP REQUEST	7	DHCP RELEASE
4	DHCP DECLINE	8	DHCP INFORM



Operation of DHCP



1. The joining host creates a **DHCP**DISCOVER message in which only the transaction-ID field is set to a random number. No other field can be set because the host has no knowledge with which to do so. This message is encapsulated in a UDP user datagram with the source port set to 68 and the destination port set to 67. We will discuss the reason for using two well-known port numbers later. The user datagram is encapsulated in an IP datagram with the source address set to **0.0.0.0** (“this host”) and the destination address set to **255.255.255.255** (broadcast address). The reason is that the joining host knows neither its own address nor the server address.

2. The DHCP server or servers (if more than one) responds with a **DHCP**OFFER message in which the your address field defines the offered IP address for the joining host and the server address field includes the IP address of the server. The message also includes the lease time for which the host can keep the IP address. This message is encapsulated in a user datagram with the same port numbers, but in the reverse order. The user datagram in turn is encapsulated in a datagram with the server address as the source IP address, but the destination address is a broadcast address, in which the server allows other DHCP servers to receive the offer and give a better offer if they can.

3. The joining host receives one or more offers and selects the best of them. The joining host then sends a **DHCPREQUEST** message to the server that has given the best offer. The fields with known value are set. The message is encapsulated in a user datagram with port numbers as the first message. The user datagram is encapsulated in an IP datagram with the source address set to the new client address, but the destination address still is set to the broadcast address to let the other servers know that their offer was not accepted.
4. Finally, the selected server responds with a **DHCPACK** message to the client if the offered IP address is valid. If the server cannot keep its offer (for example, if the address is offered to another host in between), the server sends a **DHCPNACK** message and the client needs to repeat the process. This message is also broadcast to let other servers know that the request is accepted or rejected.

Using FTP

The server does not send all of the information that a client may need for joining the network. In the **DHCPACK** message, the server defines the pathname of a file in which the client can find complete information such as the address of the DNS server. The client can then use a file transfer protocol to obtain the rest of the needed information.

Finite State Machine (FSM) for DHCP Client

